Boiler Farm Management

Frontend API Handoff Document

Project flow, screen mapping, and full API reference with dummy payloads and responses.

Generated from live backend contract on 12 May 2026.

Coverage: 77 Swagger operations + 2 support routes (79 total routes).

Environment	Value
Local base URL	http://localhost:4000/api/v1
Production base URL	https://<railway-domain>.up.railway.app/api/v1
Swagger UI	http://localhost:4000/docs
Auth mode	Bearer JWT access token in Authorization header
Primary frontend targets	Android app first, tablet-friendly layouts, lightweight responsive web panel

1. Business Flow

1. Owner registers the organization or signs in, then the app hydrates profile, permissions, dashboard summary, and subscription state.

2. Owner configures organization rules, creates users, assigns farms, and manages role-specific permissions including finance access.

3. Operations team creates farms, maintains catalog and trader masters, and opens one or more batches under each farm.

4. Farmers and supervisors capture daily logs, treatments, comments, and batch expenses only for the farms and batches they can access.

5. Purchases move inventory in, allocation moves stock out to batches, and farmer-ledger expenses can go through approval before settlement.

6. Sales can be created as draft rows, finalized later by owner or accounts, and then moved into settlement and P&L review.

7. Finance records payments and organization ledger entries, while reports, notifications, dashboards, and exports provide audit-ready visibility.

2. Role Matrix

Role	Main UI capabilities	Backend visibility rule
OWNER	Full dashboard, users, farms, batches, master data, settings, finance, inventory, settlements, reports, exports, subscription	All organization data
ACCOUNTS	Finance dashboard, purchases, payments, expenses, settlements, P&L, reports, notifications, selected batch detail	All organization data, but no staff/farm master write APIs unless separately exposed
SUPERVISOR	Farm and batch operations, master data, review/correction, comments, expenses, selected finance entry flows, reports, exports	All organization data
FARMER	Assigned farm and batch list, daily logs, treatments, farmer expenses, comments, notifications, batch/farm summary views	Only farms assigned to the farmer or marked as primary farmer

3. Integration Conventions

Authorization header format: Authorization: Bearer <accessToken>

JSON content type: application/json for all request/response bodies except file exports and /docs.

Date handling: request payloads typically accept YYYY-MM-DD for business dates, while responses return ISO strings.

Pagination: list endpoints use page, limit, and optional search. Response meta returns page, limit, total, and totalPages.

Permissions: use GET /auth/me and user.permissions from user APIs to drive screen visibility and action-level access control.

Preferred expense API: use /batches/{batchId}/expenses for new work. /batches/{batchId}/costs is a backward-compatible alias only.

Dual-ledger behavior: company and farmer expenses are stored separately. Do not merge them on the client unless a screen explicitly needs a combined view.

Offline-friendly fields: clientReferenceId appears on expenses, daily logs, treatments, purchases, and sales to support safe mobile retries.

Validation errors return HTTP 422 with message and field details. Business-rule errors return 400/403/404/409 with message and optional details.

4. Screen To API Mapping

Screen / module	Primary endpoints
Owner onboarding / login	POST /auth/register-owner, POST /auth/login, POST /auth/login-pin, GET /auth/me
Session + profile security	POST /auth/refresh, POST /auth/logout, POST /auth/change-password, POST /auth/set-pin, POST /auth/biometric
Dashboard	GET /dashboard, GET /dashboard/financial, GET /notifications, PATCH /notifications/{notificationId}/read
User management	GET /users, POST /users, GET /users/{userId}, PUT /users/{userId}, PATCH /users/{userId}/status, POST /users/{userId}/reset-password
Farm management	GET /farms, POST /farms, GET /farms/{farmId}, PUT /farms/{farmId}
Master data	GET/POST/PUT /master-data/catalog-items, GET/POST/PUT /master-data/traders
Batch operations	GET/POST/PUT /batches, PATCH /batches/{batchId}/status
Batch execution	GET/POST /batches/{batchId}/daily-logs, PUT /batches/{batchId}/daily-logs/{dailyLogId}, GET/POST /batches/{batchId}/treatments, GET/POST /batches/{batchId}/comments
Batch expenses	GET/POST /batches/{batchId}/expenses, PUT /batches/{batchId}/expenses/{expenseId}, PATCH /batches/{batchId}/expenses/{expenseId}/approval, legacy /costs alias
Sales / settlement / P&L	GET/POST /batches/{batchId}/sales, PATCH /batches/{batchId}/sales/{saleId}/finalize, GET/POST /batches/{batchId}/settlement, GET /batches/{batchId}/pnl
Inventory	GET /inventory/ledger, POST /inventory/allocate
Finance	GET/POST/PUT /finance/purchases, GET/POST /finance/payments, GET/POST /finance/entries
Reports and exports	GET /reports/overview, GET /reports/expenses, GET /reports/inventory, GET /reports/settlements, GET /reports/profitability, farm/batch summary, export endpoints
Organization settings	GET /settings/organization, PUT /settings/organization
Subscription flow	GET /subscriptions/plans, GET /subscriptions/current, POST /subscriptions/requests, POST /subscriptions/payments

5. Detailed API Reference

Each endpoint below includes its intended UI placement, auth requirement, dummy request data, and success response sample.

5.1 Authentication

POST /api/v1/auth/biometric

Purpose: Turn biometric login on or off for the current user profile.

Implement in UI: Profile > security > biometric toggle

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

`Sample request line: POST /api/v1/auth/biometric`

Note: This updates the server-side biometricEnabled flag only.

Note: Device-level biometric enrollment still needs to be handled by the frontend/app shell.

Dummy request body

```
{
  "enabled": true
}
```

Dummy success response

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "name": "Gopal Mewada",
  "email": "owner@example.com",
  "phone": "9876543210",
  "role": "OWNER",
  "status": "ACTIVE",
  "mustChangePassword": true,
  "biometricEnabled": true,
  "permissions": {
    "dailyEntry": true,
    "salesEntry": true,
    "expenseEntry": true,
    "inventoryView": true,
    "costVisibility": true,
    "reportAccess": true,
    "companyExpenseEntry": true,
    "farmerExpenseApproval": true,
    "purchaseEntry": true,
    "settlementEntry": true,
    "financialDashboard": true
  }
}
```

POST /api/v1/auth/change-password

Purpose: Allow the current user to rotate their password.

Implement in UI: Profile > security

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

`Sample request line: POST /api/v1/auth/change-password`

Note: Requires currentPassword and newPassword.

Note: Use a dedicated password form, not the general profile screen.

Dummy request body

```
{
  "currentPassword": "StrongPass@123",
  "newPassword": "NewStrongPass@456"
}
```

Dummy success response

```
{
  "message": "Password updated successfully"
}
```

POST /api/v1/auth/login

Purpose: Authenticate by mobile number and password, then return a fresh access token plus refresh token.

Implement in UI: Login

Auth required: No

Allowed roles: Public

`Sample request line: POST /api/v1/auth/login`

Note: Use the phone field with a valid 10-digit Indian mobile number.

Note: Only ACTIVE users can log in.

Dummy request body

```
{
  "phone": "9876543210",
  "password": "StrongPass@123"
}
```

Dummy success response

```
{
  "user": {
    "id": "11111111-1111-1111-1111-111111111111",
    "organizationId": "22222222-2222-2222-2222-222222222222",
    "name": "Gopal Mewada",
    "email": "owner@example.com",
    "phone": "9876543210",
    "role": "OWNER",
    "status": "ACTIVE",
    "mustChangePassword": true,
    "biometricEnabled": true,
    "permissions": {
      "dailyEntry": true,
      "salesEntry": true,
      "expenseEntry": true,
      "inventoryView": true,
      "costVisibility": true,
      "reportAccess": true,
      "companyExpenseEntry": true,
      "farmerExpenseApproval": true,
      "purchaseEntry": true,
      "settlementEntry": true,
      "financialDashboard": true
    }
  },
  "tokens": {
    "accessToken": "sample",
    "refreshToken": "sample"
  }
}
```

POST /api/v1/auth/login-pin

Purpose: Authenticate by mobile number and 4-digit PIN for repeat mobile logins after the user has set a PIN.

Implement in UI: Login > quick PIN access

Auth required: No

Allowed roles: Public

`Sample request line: POST /api/v1/auth/login-pin`

Note: Use this only after the user has created a PIN through POST /auth/set-pin.

Note: The response shape is identical to password login.

Dummy request body

```
{
  "phone": "9876543210",
  "pin": "2580"
}
```

Dummy success response

```
{
  "user": {
    "id": "11111111-1111-1111-1111-111111111111",
    "organizationId": "22222222-2222-2222-2222-222222222222",
    "name": "Gopal Mewada",
    "email": "owner@example.com",
    "phone": "9876543210",
    "role": "OWNER",
    "status": "ACTIVE",
    "mustChangePassword": true,
    "biometricEnabled": true,
    "permissions": {
      "dailyEntry": true,
      "salesEntry": true,
      "expenseEntry": true,
      "inventoryView": true,
      "costVisibility": true,
      "reportAccess": true,
      "companyExpenseEntry": true,
      "farmerExpenseApproval": true,
      "purchaseEntry": true,
      "settlementEntry": true,
      "financialDashboard": true
    }
  },
  "tokens": {
    "accessToken": "sample",
    "refreshToken": "sample"
  }
}
```

POST /api/v1/auth/logout

Purpose: Revoke the current refresh token and end the server-side session chain.

Implement in UI: Profile / logout action

Auth required: No

Allowed roles: Public

`Sample request line: POST /api/v1/auth/logout`

Note: Call before removing local auth state.

Note: Safe to treat as idempotent on the UI side.

Dummy request body

```
{
  "refreshToken": "<refresh-token>"
}
```

Dummy success response

```
{
  "message": "Logged out successfully"
}
```

GET /api/v1/auth/me

Purpose: Fetch the signed-in user profile, role, biometric flag, and permission matrix.

Implement in UI: App bootstrap / profile header

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

`Sample request line: GET /api/v1/auth/me`

Note: Useful after login, refresh, or hard app reload.

Note: Hydrate role-based tabs and permission-guarded actions from this response instead of hardcoding them on the client.

Dummy success response

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "name": "Gopal Mewada",
  "email": "owner@example.com",
  "phone": "9876543210",
  "role": "OWNER",
  "status": "ACTIVE",
  "mustChangePassword": true,
  "biometricEnabled": true,
  "permissions": {
    "dailyEntry": true,
    "salesEntry": true,
    "expenseEntry": true,
    "inventoryView": true,
    "costVisibility": true,
    "reportAccess": true,
    "companyExpenseEntry": true,
    "farmerExpenseApproval": true,
    "purchaseEntry": true,
    "settlementEntry": true,
    "financialDashboard": true
  }
}
```

POST /api/v1/auth/refresh

Purpose: Exchange a refresh token for a new token pair without asking the user to log in again.

Implement in UI: App bootstrap / silent re-auth

Auth required: No

Allowed roles: Public

`Sample request line: POST /api/v1/auth/refresh`

Note: Use on app launch or after an access-token expiry.

Note: The previous refresh token is revoked once this call succeeds.

Dummy request body

```
{
  "refreshToken": "<refresh-token>"
}
```

Dummy success response

```
{
  "user": {
    "id": "11111111-1111-1111-1111-111111111111",
    "organizationId": "22222222-2222-2222-2222-222222222222",
    "name": "Gopal Mewada",
    "email": "owner@example.com",
    "phone": "9876543210",
    "role": "OWNER",
    "status": "ACTIVE",
    "mustChangePassword": true,
    "biometricEnabled": true,
    "permissions": {
      "dailyEntry": true,
      "salesEntry": true,
      "expenseEntry": true,
      "inventoryView": true,
      "costVisibility": true,
      "reportAccess": true,
      "companyExpenseEntry": true,
      "farmerExpenseApproval": true,
      "purchaseEntry": true,
      "settlementEntry": true,
      "financialDashboard": true
    }
  },
  "tokens": {
    "accessToken": "sample",
    "refreshToken": "sample"
  }
}
```

POST /api/v1/auth/register-owner

Purpose: Create the organization, create the owner account, auto-create a 14-day trial subscription, and return tokens.

Implement in UI: Owner onboarding

Auth required: No

Allowed roles: Public

`Sample request line: POST /api/v1/auth/register-owner`

Note: Owner phone is mandatory and must be a valid 10-digit Indian mobile number.

Note: Use this only once per new business account.

Note: Successful response can immediately take the user into the dashboard without a second login.

Dummy request body

```
{
  "organizationName": "Mewada Broiler Ventures",
  "organizationPhone": "9876543210",
  "organizationEmail": "accounts@mewadabroiler.com",
  "ownerName": "Gopal Mewada",
  "ownerEmail": "owner@example.com",
```

```
  "ownerPhone": "9876543210",
  "password": "StrongPass@123"
}
```

Dummy success response

```
{
  "user": {
    "id": "11111111-1111-1111-1111-111111111111",
    "organizationId": "22222222-2222-2222-2222-222222222222",
    "name": "Gopal Mewada",
    "email": "owner@example.com",
    "phone": "9876543210",
    "role": "OWNER",
    "status": "ACTIVE",
    "mustChangePassword": true,
    "biometricEnabled": true,
    "permissions": {
      "dailyEntry": true,
      "salesEntry": true,
      "expenseEntry": true,
      "inventoryView": true,
      "costVisibility": true,
      "reportAccess": true,
      "companyExpenseEntry": true,
      "farmerExpenseApproval": true,
      "purchaseEntry": true,
      "settlementEntry": true,
      "financialDashboard": true
    }
  },
  "tokens": {
    "accessToken": "sample",
    "refreshToken": "sample"
  }
}
```

POST /api/v1/auth/set-pin

Purpose: Create or replace the mobile quick-login PIN for the signed-in user.

Implement in UI: Profile > security > set PIN

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

`Sample request line: POST /api/v1/auth/set-pin`

Note: Requires the current account password for confirmation.

Note: Use this before enabling PIN login on shared devices.

Dummy request body

```
{
  "currentPassword": "StrongPass@123",
  "pin": "2580"
}
```

Dummy success response

```
{
  "message": "Operation completed successfully"
}
```

5.2 Dashboard

GET /api/v1/dashboard

Purpose: Load operational summary cards, active batch tiles, payment buckets, and alert banners for the signed-in user.

Implement in UI: Home dashboard

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

`Sample request line: GET /api/v1/dashboard`

Note: This is the main landing API after login.

Note: Farmer visibility is already restricted to assigned farms and batches.

Dummy success response

```
{
  "farmCount": 4,
  "today": {
    "activeBatches": 1,
    "liveBirds": 4876,
    "mortalityToday": 1,
    "mortalityTotal": 124,
    "salesReady": 1,
    "pendingEntries": 2,
    "feedAlert": 1,
    "fcrAlert": 1
  },
  "activeBatches": [
    {
      "batchId": "55555555-5555-5555-5555-555555555555",
      "batchCode": "BATCH-MAY-2026-01",
      "farmId": "44444444-4444-4444-4444-444444444444",
      "farmName": "Shivam Farm",
      "status": "ACTIVE",
      "currentAgeDays": 31,
      "liveBirds": 4876,
      "mortalityPercent": 2.48
    }
  ],
  "paymentStatus": {
    "pending": 3,
    "partial": 2,
    "paid": 8
  },
  "alerts": [
    {
      "id": "12121212-1212-1212-1212-121212121212",
      "type": "FEED_ALERT",
      "severity": "WARNING",
      "title": "Feed stock running low",
      "message": "Starter Feed stock is below the configured reorder level.",
      "batchId": "55555555-5555-5555-5555-555555555555",
      "farmId": "44444444-4444-4444-4444-444444444444",
      "createdAt": "2026-05-12T07:18:15.474Z"
    }
  ]
}
```

GET /api/v1/dashboard/financial

Purpose: Load owner/accounts-level financial summary, recent transactions, and payment buckets.

Implement in UI: Finance dashboard

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS

`Sample request line: GET /api/v1/dashboard/financial`

Note: Use this for finance-only cards and cashflow widgets.

Note: Do not show this module unless the user role and permission set allow it.

Dummy success response

```
{
  "summary": {
    "investment": 1250000,
    "expenses": 482500,
    "sales": 845000,
    "netProfitOrLoss": 362500
  },
  "recentTransactions": [
    {
      "id": "ffffffff-ffff-ffff-ffff-ffffffffffff",
      "type": "PURCHASE",
      "direction": "OUTBOUND",
      "amount": 45000,
      "date": "2026-05-12",
      "description": "Starter feed purchase installment"
    },
    {
      "id": "eeeeeeee-eeee-eeee-eeee-eeeeeeeeeeee",
      "type": "SALE_RECEIPT",
      "direction": "INBOUND",
      "amount": 225000,
      "date": "2026-05-12",
      "description": "Trader receipt against Batch MAY-2026-01"
    }
  ],
  "paymentStatus": {
    "pending": 3,
    "partial": 2,
    "paid": 8
  }
}
```

5.3 Users

GET /api/v1/users

Purpose: Load user records, role badges, permissions, and farm assignments for the organization.

Implement in UI: Settings > users

Auth required: Yes

Allowed roles: OWNER, SUPERVISOR

`Sample request line: GET /api/v1/users?page=1&limit=20&search=shivam`

Note: List/read access is available to OWNER and SUPERVISOR; write actions are owner-only unless explicitly stated otherwise.

Note: Use role, status, biometricEnabled, mustChangePassword, and permissions fields to drive UI states.

Parameters

Name	In	Required	Example	Type / notes
page	query	No	1	number
limit	query	No	20	number

search	query	No	shivam	string

Dummy success response

```
{
  "data": [
    {
      "id": "33333333-3333-3333-3333-333333333333",
      "organizationId": "22222222-2222-2222-2222-222222222222",
      "name": "Ramesh Patel",
      "email": "ramesh.patel@example.com",
      "phone": "9876500001",
      "role": "FARMER",
      "status": "ACTIVE",
      "mustChangePassword": true,
      "biometricEnabled": false,
      "permissions": {
        "dailyEntry": true,
        "salesEntry": false,
        "expenseEntry": true,
        "inventoryView": true,
        "costVisibility": false,
        "reportAccess": true,
        "companyExpenseEntry": false,
        "farmerExpenseApproval": false,
        "purchaseEntry": false,
        "settlementEntry": false,
        "financialDashboard": false
      },
      "assignedFarmIds": [
        "44444444-4444-4444-4444-444444444444"
      ],
      "lastLoginAt": "2026-05-12T07:18:15.474Z",
      "createdAt": "2026-05-12T07:18:15.474Z",
      "updatedAt": "2026-05-12T07:18:15.474Z"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 1,
    "totalPages": 1
  }
}
```

POST /api/v1/users

Purpose: Maintain staff accounts, permissions, login rules, and farm assignments.

Implement in UI: Settings > users

Auth required: Yes

Allowed roles: OWNER

`Sample request line: POST /api/v1/users`

Note: List/read access is available to OWNER and SUPERVISOR; write actions are owner-only unless explicitly stated otherwise.

Note: CreateUser requires a phone number, role, password, and can optionally include permissions plus assignedFarmIds.

Dummy request body

```
{
  "name": "Ramesh Patel",
  "email": "ramesh.patel@example.com",
```

```
  "phone": "9876500001",
  "password": "FarmerPass@123",
  "role": "FARMER",
  "assignedFarmIds": [
    "44444444-4444-4444-4444-444444444444"
  ],
  "permissions": {
    "dailyEntry": true,
    "expenseEntry": true,
    "inventoryView": true,
    "reportAccess": true
  },
  "mustChangePassword": true
}
```

Dummy success response

```
{
  "id": "33333333-3333-3333-3333-333333333333",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "name": "Ramesh Patel",
  "email": "ramesh.patel@example.com",
  "phone": "9876500001",
  "role": "FARMER",
  "status": "ACTIVE",
  "mustChangePassword": true,
  "biometricEnabled": false,
  "permissions": {
    "dailyEntry": true,
    "salesEntry": false,
    "expenseEntry": true,
    "inventoryView": true,
    "costVisibility": false,
    "reportAccess": true,
    "companyExpenseEntry": false,
    "farmerExpenseApproval": false,
    "purchaseEntry": false,
    "settlementEntry": false,
    "financialDashboard": false
  },
  "assignedFarmIds": [
    "44444444-4444-4444-4444-444444444444"
  ],
  "lastLoginAt": null,
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```

GET /api/v1/users/{userId}

Purpose: Load user records, role badges, permissions, and farm assignments for the organization.

Implement in UI: Settings > users

Auth required: Yes

Allowed roles: OWNER, SUPERVISOR

`Sample request line: GET /api/v1/users/33333333-3333-3333-3333-333333333333`

Note: List/read access is available to OWNER and SUPERVISOR; write actions are owner-only unless explicitly stated otherwise.

Note: Use role, status, biometricEnabled, mustChangePassword, and permissions fields to drive UI states.

Parameters

Name	In	Required	Example	Type / notes

userId	path	Yes	33333333-3333-3333-3333-333333333333	string

Dummy success response

```
{
  "id": "33333333-3333-3333-3333-333333333333",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "name": "Ramesh Patel",
  "email": "ramesh.patel@example.com",
  "phone": "9876500001",
  "role": "FARMER",
  "status": "ACTIVE",
  "mustChangePassword": true,
  "biometricEnabled": false,
  "permissions": {
    "dailyEntry": true,
    "salesEntry": false,
    "expenseEntry": true,
    "inventoryView": true,
    "costVisibility": false,
    "reportAccess": true,
    "companyExpenseEntry": false,
    "farmerExpenseApproval": false,
    "purchaseEntry": false,
    "settlementEntry": false,
    "financialDashboard": false
  },
  "assignedFarmIds": [
    "44444444-4444-4444-4444-444444444444"
  ],
  "lastLoginAt": "2026-05-12T07:18:15.474Z",
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```

PUT /api/v1/users/{userId}

Purpose: Maintain staff accounts, permissions, login rules, and farm assignments.

Implement in UI: Settings > users

Auth required: Yes

Allowed roles: OWNER

`Sample request line: PUT /api/v1/users/33333333-3333-3333-3333-333333333333`

Note: List/read access is available to OWNER and SUPERVISOR; write actions are owner-only unless explicitly stated otherwise.

Note: Use role, status, biometricEnabled, mustChangePassword, and permissions fields to drive UI states.

Parameters

Name	In	Required	Example	Type / notes
userId	path	Yes	33333333-3333-3333-3333-333333333333	string

Dummy request body

```
{
  "name": "Ramesh Patel Updated",
  "phone": "9876500002",
  "role": "FARMER",
  "assignedFarmIds": [
    "44444444-4444-4444-4444-444444444444"
```

```
  ],
  "biometricEnabled": true,
  "mustChangePassword": false
}
```

Dummy success response

```
{
  "id": "33333333-3333-3333-3333-333333333333",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "name": "Ramesh Patel Updated",
  "email": "ramesh.patel@example.com",
  "phone": "9876500002",
  "role": "FARMER",
  "status": "ACTIVE",
  "mustChangePassword": false,
  "biometricEnabled": true,
  "permissions": {
    "dailyEntry": true,
    "salesEntry": false,
    "expenseEntry": true,
    "inventoryView": true,
    "costVisibility": false,
    "reportAccess": true,
    "companyExpenseEntry": false,
    "farmerExpenseApproval": false,
    "purchaseEntry": false,
    "settlementEntry": false,
    "financialDashboard": false
  },
  "assignedFarmIds": [
    "44444444-4444-4444-4444-444444444444"
  ],
  "lastLoginAt": "2026-05-12T07:18:15.474Z",
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```

POST /api/v1/users/{userId}/reset-password

Purpose: Force-reset another user's password and optionally mark it for mandatory change on next login.

Implement in UI: Settings > users > reset password

Auth required: Yes

Allowed roles: OWNER

`Sample request line: POST /api/v1/users/33333333-3333-3333-3333-333333333333/reset-password`

Note: This is an owner-only administrative flow.

Note: Use it when replacing devices, onboarding new staff, or recovering locked accounts.

Parameters

Name	In	Required	Example	Type / notes
userId	path	Yes	33333333-3333-3333-3333-333333333333	string

Dummy request body

```
{
  "newPassword": "TempPass@456",
  "mustChangePassword": true
}
```

Dummy success response

```
{
```

```
  "message": "Operation completed successfully"
}
```

PATCH /api/v1/users/{userId}/status

Purpose: Maintain staff accounts, permissions, login rules, and farm assignments.

Implement in UI: Settings > users

Auth required: Yes

Allowed roles: OWNER

`Sample request line: PATCH /api/v1/users/33333333-3333-3333-3333-333333333333/status`

Note: List/read access is available to OWNER and SUPERVISOR; write actions are owner-only unless explicitly stated otherwise.

Note: Use role, status, biometricEnabled, mustChangePassword, and permissions fields to drive UI states.

Parameters

Name	In	Required	Example	Type / notes
userId	path	Yes	33333333-3333-3333-3333-333333333333	string

Dummy request body

```
{
  "status": "DISABLED"
}
```

Dummy success response

```
{
  "id": "33333333-3333-3333-3333-333333333333",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "name": "Ramesh Patel",
  "email": "ramesh.patel@example.com",
  "phone": "9876500001",
  "role": "FARMER",
  "status": "DISABLED",
  "mustChangePassword": true,
  "biometricEnabled": false,
  "permissions": {
    "dailyEntry": true,
    "salesEntry": false,
    "expenseEntry": true,
    "inventoryView": true,
    "costVisibility": false,
    "reportAccess": true,
    "companyExpenseEntry": false,
    "farmerExpenseApproval": false,
    "purchaseEntry": false,
    "settlementEntry": false,
    "financialDashboard": false
  },
  "assignedFarmIds": [
    "44444444-4444-4444-4444-444444444444"
  ],
  "lastLoginAt": "2026-05-12T07:18:15.474Z",
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```

5.4 Farms

GET /api/v1/farms

Purpose: Fetch farm records visible to the current user.

Implement in UI: Farm list / farm detail / farm form

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

`Sample request line: GET /api/v1/farms?page=1&limit=20&search=shivam`

Note: Farmers only see farms they are assigned to or marked as primary farmer for.

Note: Use activeBatchCount for badges on the list or farm cards.

Parameters

Name	In	Required	Example	Type / notes
page	query	No	1	number
limit	query	No	20	number
search	query	No	shivam	string

Dummy success response

```
{
  "data": [
    {
      "id": "11111111-1111-1111-1111-111111111111",
      "organizationId": "22222222-2222-2222-2222-222222222222",
      "name": "Shivam Farm",
      "code": "FARM-001",
      "location": "Near Main Road",
      "village": "Rampura",
      "district": "Indore",
      "state": "Madhya Pradesh",
      "capacity": 1,
      "status": "ACTIVE",
      "notes": "Captured from mobile app",
      "primaryFarmerId": "33333333-3333-3333-3333-333333333333",
      "supervisorId": "sample",
      "primaryFarmerName": "Ramesh Patel",
      "supervisorName": "Shivam Farm",
      "assignments": [
        {
          "userId": "33333333-3333-3333-3333-333333333333",
          "name": "Shivam Farm",
          "role": "OWNER"
        }
      ],
      "activeBatchCount": 1,
      "createdAt": "2026-05-12T07:18:15.474Z",
      "updatedAt": "2026-05-12T07:18:15.474Z"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 1,
    "totalPages": 1
  }
}
```

POST /api/v1/farms

Purpose: Create or edit farm master data and assignments.

Implement in UI: Farm list / farm detail / farm form

Auth required: Yes

Allowed roles: OWNER, SUPERVISOR

`Sample request line: POST /api/v1/farms`

Note: Farmers only see farms they are assigned to or marked as primary farmer for.

Note: assignmentUserIds is automatically merged with primaryFarmerId and supervisorId.

Dummy request body

```
{
  "name": "Shivam Farm",
  "code": "FARM-001",
  "location": "Near Main Road",
  "village": "Rampura",
  "district": "Indore",
  "state": "Madhya Pradesh",
  "capacity": 7000,
  "notes": "Primary contract farm",
  "primaryFarmerId": "33333333-3333-3333-3333-333333333333",
  "supervisorId": "eeeeeeee-eeee-eeee-eeee-eeeeeeeeeeee",
  "assignmentUserIds": [
    "33333333-3333-3333-3333-333333333333",
    "eeeeeeee-eeee-eeee-eeee-eeeeeeeeeeee"
  ]
}
```

Dummy success response

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "name": "Shivam Farm",
  "code": "FARM-001",
  "location": "Near Main Road",
  "village": "Rampura",
  "district": "Indore",
  "state": "Madhya Pradesh",
  "capacity": 1,
  "status": "ACTIVE",
  "notes": "Captured from mobile app",
  "primaryFarmerId": "33333333-3333-3333-3333-333333333333",
  "supervisorId": "sample",
  "primaryFarmerName": "Ramesh Patel",
  "supervisorName": "Shivam Farm",
  "assignments": [
    {
      "userId": "33333333-3333-3333-3333-333333333333",
      "name": "Shivam Farm",
      "role": "OWNER"
    }
  ],
  "activeBatchCount": 1,
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```

GET /api/v1/farms/{farmId}

Purpose: Fetch farm records visible to the current user.

Implement in UI: Farm list / farm detail / farm form

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

`Sample request line: GET /api/v1/farms/44444444-4444-4444-4444-444444444444`

Note: Farmers only see farms they are assigned to or marked as primary farmer for.

Note: Use activeBatchCount for badges on the list or farm cards.

Parameters

Name	In	Required	Example	Type / notes
farmId	path	Yes	44444444-4444-4444-4444-444444444444	string

Dummy success response

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "name": "Shivam Farm",
  "code": "FARM-001",
  "location": "Near Main Road",
  "village": "Rampura",
  "district": "Indore",
  "state": "Madhya Pradesh",
  "capacity": 1,
  "status": "ACTIVE",
  "notes": "Captured from mobile app",
  "primaryFarmerId": "33333333-3333-3333-3333-333333333333",
  "supervisorId": "sample",
  "primaryFarmerName": "Ramesh Patel",
  "supervisorName": "Shivam Farm",
  "assignments": [
    {
      "userId": "33333333-3333-3333-3333-333333333333",
      "name": "Shivam Farm",
      "role": "OWNER"
    }
  ],
  "activeBatchCount": 1,
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```

PUT /api/v1/farms/{farmId}

Purpose: Create or edit farm master data and assignments.

Implement in UI: Farm list / farm detail / farm form

Auth required: Yes

Allowed roles: OWNER, SUPERVISOR

`Sample request line: PUT /api/v1/farms/44444444-4444-4444-4444-444444444444`

Note: Farmers only see farms they are assigned to or marked as primary farmer for.

Note: assignmentUserIds is automatically merged with primaryFarmerId and supervisorId.

Parameters

Name	In	Required	Example	Type / notes
farmId	path	Yes	44444444-4444-4444-4444-444444444444	string

Dummy request body

```
{
  "name": "Shivam Farm",
  "code": "FARM-001",
  "location": "Near Main Road",
  "village": "Rampura",
  "district": "Indore",
  "state": "Madhya Pradesh",
  "capacity": 1,
  "notes": "Captured from mobile app",
  "status": "ACTIVE",
  "primaryFarmerId": "33333333-3333-3333-3333-333333333333",
  "supervisorId": "sample",
  "assignmentUserIds": [
    "sample"
  ]
}
```

Dummy success response

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "name": "Shivam Farm",
  "code": "FARM-001",
  "location": "Near Main Road",
  "village": "Rampura",
  "district": "Indore",
  "state": "Madhya Pradesh",
  "capacity": 1,
  "status": "ACTIVE",
  "notes": "Captured from mobile app",
  "primaryFarmerId": "33333333-3333-3333-3333-333333333333",
  "supervisorId": "sample",
  "primaryFarmerName": "Ramesh Patel",
  "supervisorName": "Shivam Farm",
  "assignments": [
    {
      "userId": "33333333-3333-3333-3333-333333333333",
      "name": "Shivam Farm",
      "role": "OWNER"
    }
  ],
  "activeBatchCount": 1,
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```

5.5 Master Data

GET /api/v1/master-data/catalog-items

Purpose: Manage feed, vaccine, medicine, chicks, equipment, and other reusable catalog items.

Implement in UI: Settings > catalog master

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

`Sample request line: GET /api/v1/master-data/catalog-items?page=1&limit=20&search=shivam&type=sample`

Note: Catalog items are referenced by expenses, treatments, purchases, and inventory allocations.

Note: Supports pagination, search, and optional type filter for catalog items.

Parameters

Name	In	Required	Example	Type / notes

page	query	No	1	number
limit	query	No	20	number
search	query	No	shivam	string
type	query	No	sample	string

Dummy success response

```
{
  "data": [
    {
      "id": "11111111-1111-1111-1111-111111111111",
      "organizationId": "22222222-2222-2222-2222-222222222222",
      "type": "FEED",
      "name": "Starter Feed",
      "sku": "sample",
      "unit": "kg",
      "defaultRate": 36,
      "manufacturer": "AgroVet",
      "reorderLevel": 500,
      "currentStock": 1380,
      "isActive": true,
      "createdAt": "2026-05-12T07:18:15.474Z",
      "updatedAt": "2026-05-12T07:18:15.474Z"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 1,
    "totalPages": 1
  }
}
```

POST /api/v1/master-data/catalog-items

Purpose: Manage feed, vaccine, medicine, chicks, equipment, and other reusable catalog items.

Implement in UI: Settings > catalog master

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR

`Sample request line: POST /api/v1/master-data/catalog-items`

Note: Catalog items are referenced by expenses, treatments, purchases, and inventory allocations.

Note: Keep these masters clean so mobile forms can use dropdowns or autocomplete.

Dummy request body

```
{
  "type": "FEED",
  "name": "Starter Feed",
  "sku": "sample",
  "unit": "kg",
  "defaultRate": 36,
  "manufacturer": "AgroVet",
  "reorderLevel": 500,
  "currentStock": 1380
}
```

Dummy success response

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
```

```
  "type": "FEED",
  "name": "Starter Feed",
  "sku": "sample",
  "unit": "kg",
  "defaultRate": 36,
  "manufacturer": "AgroVet",
  "reorderLevel": 500,
  "currentStock": 1380,
  "isActive": true,
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```

PUT /api/v1/master-data/catalog-items/{itemId}

Purpose: Manage feed, vaccine, medicine, chicks, equipment, and other reusable catalog items.

Implement in UI: Settings > catalog master

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR

`Sample request line: PUT /api/v1/master-data/catalog-items/cccccccc-cccc-cccc-cccc-cccccccccccc`

Note: Catalog items are referenced by expenses, treatments, purchases, and inventory allocations.

Note: Keep these masters clean so mobile forms can use dropdowns or autocomplete.

Parameters

Name	In	Required	Example	Type / notes
itemId	path	Yes	cccccccc-cccc-cccc-cccc-cccccccccccc	string

Dummy request body

```
{
  "name": "Starter Feed",
  "sku": "sample",
  "unit": "kg",
  "defaultRate": 36,
  "manufacturer": "AgroVet",
  "reorderLevel": 500,
  "currentStock": 1380,
  "isActive": true
}
```

Dummy success response

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "type": "FEED",
  "name": "Starter Feed",
  "sku": "sample",
  "unit": "kg",
  "defaultRate": 36,
  "manufacturer": "AgroVet",
  "reorderLevel": 500,
  "currentStock": 1380,
  "isActive": true,
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```

GET /api/v1/master-data/traders

Purpose: Manage trader master data used during sale entry.

Implement in UI: Settings > traders

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

`Sample request line: GET /api/v1/master-data/traders?page=1&limit=20&search=shivam`

Note: Traders are referenced by sales entries.

Note: Supports pagination, search, and optional type filter for catalog items.

Parameters

Name	In	Required	Example	Type / notes
page	query	No	1	number
limit	query	No	20	number
search	query	No	shivam	string

Dummy success response

```
{
  "data": [
    {
      "id": "11111111-1111-1111-1111-111111111111",
      "organizationId": "22222222-2222-2222-2222-222222222222",
      "name": "Mahadev Traders",
      "phone": "9876543210",
      "email": "trader@example.com",
      "address": "Ward 3, Rampura",
      "notes": "Captured from mobile app",
      "createdAt": "2026-05-12T07:18:15.474Z",
      "updatedAt": "2026-05-12T07:18:15.474Z"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 1,
    "totalPages": 1
  }
}
```

POST /api/v1/master-data/traders

Purpose: Manage trader master data used during sale entry.

Implement in UI: Settings > traders

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR

`Sample request line: POST /api/v1/master-data/traders`

Note: Traders are referenced by sales entries.

Note: Keep these masters clean so mobile forms can use dropdowns or autocomplete.

Dummy request body

```
{
  "name": "Mahadev Traders",
  "phone": "9876543210",
  "email": "owner@example.com",
```

```
  "address": "Ward 3, Rampura",
  "notes": "Captured from mobile app"
}
```

Dummy success response

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "name": "Mahadev Traders",
  "phone": "9876543210",
  "email": "trader@example.com",
  "address": "Ward 3, Rampura",
  "notes": "Captured from mobile app",
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```

PUT /api/v1/master-data/traders/{traderId}

Purpose: Manage trader master data used during sale entry.

Implement in UI: Settings > traders

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR

`Sample request line: PUT /api/v1/master-data/traders/77777777-7777-7777-7777-777777777777`

Note: Traders are referenced by sales entries.

Note: Keep these masters clean so mobile forms can use dropdowns or autocomplete.

Parameters

Name	In	Required	Example	Type / notes
traderId	path	Yes	77777777-7777-7777-7777-777777777777	string

Dummy request body

```
{
  "name": "Mahadev Traders",
  "phone": "9876543210",
  "email": "owner@example.com",
  "address": "Ward 3, Rampura",
  "notes": "Captured from mobile app"
}
```

Dummy success response

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "name": "Mahadev Traders",
  "phone": "9876543210",
  "email": "trader@example.com",
  "address": "Ward 3, Rampura",
  "notes": "Captured from mobile app",
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```

5.6 Batches

GET /api/v1/batches

Purpose: Load batch records with farm and status context.

Implement in UI: Batch list / batch detail / batch form

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

```
Sample request line: GET /api/v1/batches?page=1&limit=20&search=shivam&farmId=44444444-4444-4444-4444-444444444444&status=ACTIVE
```

Note: Use status to drive lifecycle actions such as active work, sales running, settlement pending, and closed.

Note: Farmers only see batches for farms they can access.

Parameters

Name	In	Required	Example	Type / notes
page	query	No	1	number
limit	query	No	20	number
search	query	No	shivam	string
farmId	query	No	44444444-4444-4444-4444-444444444444	string
status	query	No	ACTIVE	string

Dummy success response


```
{
  "data": [
    {
      "id": "11111111-1111-1111-1111-111111111111",
      "organizationId": "22222222-2222-2222-2222-222222222222",
      "farmId": "44444444-4444-4444-4444-444444444444",
      "farmName": "Shivam Farm",
      "code": "BATCH-MAY-2026-01",
      "placementDate": "2026-05-12",
      "placementCount": 5000,
      "totalChicksPurchased": 5050,
      "freeChicks": 50,
      "chargeableChicks": 5000,
      "placementMortality": 12,
      "chickCostTotal": 220000,
      "chickRatePerBird": 44,
      "ratePerChick": 1,
      "chickTransportCharge": 5500,
      "sourceHatchery": "Sunrise Hatchery",
      "vendorName": "Sample Name",
      "targetCloseDate": "2026-05-12",
      "actualCloseDate": "2026-05-12",
      "status": "ACTIVE",
      "lockedAt": "2026-05-12T07:18:15.474Z",
      "notes": "Captured from mobile app",
      "summary": {
        "currentAgeDays": 31,
        "liveBirds": 4876,
        "mortalityCount": 8,
        "cullCount": 3,
        "mortalityPercent": 1,
        "soldBirds": 2480,
        "totalFeedConsumedKg": 1480.5,
        "totalWeightSoldKg": 2350,
        "averageWeightGrams": 2350,
        "fcr": 1.68
      },
      "createdAt": "2026-05-12T07:18:15.474Z",
```

```
      "updatedAt": "2026-05-12T07:18:15.474Z"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 1,
    "totalPages": 1
  }
}
```


POST /api/v1/batches

Purpose: Create, edit, or change the lifecycle status of a batch.

Implement in UI: Batch list / batch detail / batch form

Auth required: Yes

Allowed roles: OWNER, SUPERVISOR

`Sample request line: POST /api/v1/batches`

Note: Use status to drive lifecycle actions such as active work, sales running, settlement pending, and closed.

Note: Farmers only see batches for farms they can access.

Dummy request body


```
{
  "farmId": "44444444-4444-4444-4444-444444444444",
  "code": "BATCH-MAY-2026-01",
  "placementDate": "2026-05-12",
  "placementCount": 5000,
  "totalChicksPurchased": 5050,
  "freeChicks": 50,
  "chargeableChicks": 5000,
  "placementMortality": 12,
  "chickCostTotal": 220000,
  "chickRatePerBird": 44,
  "chickTransportCharge": 5500,
  "sourceHatchery": "Sunrise Hatchery",
  "vendorName": "Sunrise Hatchery",
  "targetCloseDate": "2026-06-10",
  "notes": "New grow-out batch"
}
```


Dummy success response


```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "farmId": "44444444-4444-4444-4444-444444444444",
  "farmName": "Shivam Farm",
  "code": "BATCH-MAY-2026-01",
  "placementDate": "2026-05-12",
  "placementCount": 5000,
  "totalChicksPurchased": 5050,
  "freeChicks": 50,
  "chargeableChicks": 5000,
  "placementMortality": 12,
  "chickCostTotal": 220000,
  "chickRatePerBird": 44,
  "ratePerChick": 1,
  "chickTransportCharge": 5500,
  "sourceHatchery": "Sunrise Hatchery",
  "vendorName": "Sample Name",
  "targetCloseDate": "2026-05-12",
  "actualCloseDate": "2026-05-12",
```

```
  "status": "ACTIVE",
  "lockedAt": "2026-05-12T07:18:15.474Z",
  "notes": "Captured from mobile app",
  "summary": {
    "currentAgeDays": 31,
    "liveBirds": 4876,
    "mortalityCount": 8,
    "cullCount": 3,
    "mortalityPercent": 1,
    "soldBirds": 2480,
    "totalFeedConsumedKg": 1480.5,
    "totalWeightSoldKg": 2350,
    "averageWeightGrams": 2350,
    "fcr": 1.68
  },
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```


GET /api/v1/batches/{batchId}

Purpose: Load batch records with farm and status context.

Implement in UI: Batch list / batch detail / batch form

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

`Sample request line: GET /api/v1/batches/55555555-5555-5555-5555-555555555555`

Note: Use status to drive lifecycle actions such as active work, sales running, settlement pending, and closed.

Note: Farmers only see batches for farms they can access.

Parameters

Name	In	Required	Example	Type / notes
batchId	path	Yes	55555555-5555-5555-5555-555555555555	string

Dummy success response


```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "farmId": "44444444-4444-4444-4444-444444444444",
  "farmName": "Shivam Farm",
  "code": "BATCH-MAY-2026-01",
  "placementDate": "2026-05-12",
  "placementCount": 5000,
  "totalChicksPurchased": 5050,
  "freeChicks": 50,
  "chargeableChicks": 5000,
  "placementMortality": 12,
  "chickCostTotal": 220000,
  "chickRatePerBird": 44,
  "ratePerChick": 1,
  "chickTransportCharge": 5500,
  "sourceHatchery": "Sunrise Hatchery",
  "vendorName": "Sample Name",
  "targetCloseDate": "2026-05-12",
  "actualCloseDate": "2026-05-12",
  "status": "ACTIVE",
  "lockedAt": "2026-05-12T07:18:15.474Z",
  "notes": "Captured from mobile app",
  "summary": {
    "currentAgeDays": 31,
```

```
    "liveBirds": 4876,
    "mortalityCount": 8,
    "cullCount": 3,
    "mortalityPercent": 1,
    "soldBirds": 2480,
    "totalFeedConsumedKg": 1480.5,
    "totalWeightSoldKg": 2350,
    "averageWeightGrams": 2350,
    "fcr": 1.68
  },
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```


PUT /api/v1/batches/{batchId}

Purpose: Create, edit, or change the lifecycle status of a batch.

Implement in UI: Batch list / batch detail / batch form

Auth required: Yes

Allowed roles: OWNER, SUPERVISOR

`Sample request line: PUT /api/v1/batches/55555555-5555-5555-5555-555555555555`

Note: Use status to drive lifecycle actions such as active work, sales running, settlement pending, and closed.

Note: Farmers only see batches for farms they can access.

Parameters

Name	In	Required	Example	Type / notes
batchId	path	Yes	55555555-5555-5555-5555-555555555555	string

Dummy request body


```
{
  "code": "BATCH-MAY-2026-01",
  "placementDate": "2026-05-12",
  "placementCount": 5000,
  "totalChicksPurchased": 5050,
  "freeChicks": 50,
  "chargeableChicks": 5000,
  "placementMortality": 12,
  "chickCostTotal": 220000,
  "chickRatePerBird": 44,
  "ratePerChick": 1,
  "chickTransportCharge": 5500,
  "sourceHatchery": "Sunrise Hatchery",
  "vendorName": "Sample Name",
  "targetCloseDate": "2026-05-12",
  "actualCloseDate": "2026-05-12",
  "notes": "Captured from mobile app"
}
```


Dummy success response


```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "farmId": "44444444-4444-4444-4444-444444444444",
  "farmName": "Shivam Farm",
  "code": "BATCH-MAY-2026-01",
  "placementDate": "2026-05-12",
  "placementCount": 5000,
  "totalChicksPurchased": 5050,
  "freeChicks": 50,
```

```
  "chargeableChicks": 5000,
  "placementMortality": 12,
  "chickCostTotal": 220000,
  "chickRatePerBird": 44,
  "ratePerChick": 1,
  "chickTransportCharge": 5500,
  "sourceHatchery": "Sunrise Hatchery",
  "vendorName": "Sample Name",
  "targetCloseDate": "2026-05-12",
  "actualCloseDate": "2026-05-12",
  "status": "ACTIVE",
  "lockedAt": "2026-05-12T07:18:15.474Z",
  "notes": "Captured from mobile app",
  "summary": {
    "currentAgeDays": 31,
    "liveBirds": 4876,
    "mortalityCount": 8,
    "cullCount": 3,
    "mortalityPercent": 1,
    "soldBirds": 2480,
    "totalFeedConsumedKg": 1480.5,
    "totalWeightSoldKg": 2350,
    "averageWeightGrams": 2350,
    "fcr": 1.68
  },
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```


GET /api/v1/batches/{batchId}/comments

Purpose: Load review comments.

Implement in UI: Batch detail > comments / corrections

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

`Sample request line: GET /api/v1/batches/55555555-5555-5555-5555-555555555555/comments`

Note: Comments can target farm, batch, daily log, treatment, cost, sale, settlement, purchase, or payment.

Note: This is useful for supervisor audit trails without overwriting the original data context.

Parameters

Name	In	Required	Example	Type / notes
batchId	path	Yes	55555555-5555-5555-5555-555555555555	string

Dummy success response


```
{
  "data": [
    {
      "id": "11111111-1111-1111-1111-111111111111",
      "organizationId": "22222222-2222-2222-2222-222222222222",
      "farmId": "44444444-4444-4444-4444-444444444444",
      "batchId": "55555555-5555-5555-5555-555555555555",
      "targetType": "DAILY_LOG",
      "targetId": "bbbbbbbb-bbbb-bbbb-bbbb-bbbbbbbbbbbb",
      "comment": "Mortality corrected after supervisor review.",
      "correctionNote": "Mortality corrected after supervisor review.",
      "createdById": "Mortality corrected after supervisor review.",
      "createdAt": "Mortality corrected after supervisor review."
    }
  ]
```

```
}
```

POST /api/v1/batches/{batchId}/comments

Purpose: Attach comments or correction notes to the farm, batch, daily log, treatment, cost, sale, settlement, purchase, or payment.

Implement in UI: Batch detail > comments / review notes

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

`Sample request line: POST /api/v1/batches/55555555-5555-5555-5555-555555555555/comments`

Note: targetType determines what targetId must point to.

Note: Use this for supervisor feedback and audit-friendly explanations.

Parameters

Name	In	Required	Example	Type / notes
batchId	path	Yes	55555555-5555-5555-5555-555555555555	string

Dummy request body

```
{
  "targetType": "DAILY_LOG",
  "targetId": "88888888-8888-8888-8888-888888888888",
  "comment": "Mortality updated after phone verification.",
  "correctionNote": "Keep source slip photo for audit."
}
```

Dummy success response

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "farmId": "44444444-4444-4444-4444-444444444444",
  "batchId": "55555555-5555-5555-5555-555555555555",
  "targetType": "DAILY_LOG",
  "targetId": "bbbbbbbb-bbbb-bbbb-bbbb-bbbbbbbbbbbb",
  "comment": "Mortality corrected after supervisor review.",
  "correctionNote": "Mortality corrected after supervisor review.",
  "createdById": "Mortality corrected after supervisor review.",
  "createdAt": "Mortality corrected after supervisor review."
}
```

GET /api/v1/batches/{batchId}/costs

Purpose: Backward-compatible alias of the batch expense list endpoint.

Implement in UI: Batch detail > expenses (legacy alias)

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

`Sample request line: GET /api/v1/batches/55555555-5555-5555-5555-555555555555/costs`

Note: Use /expenses for all new frontend implementation.

Note: This route exists to avoid breaking older clients during migration.

Parameters

Name	In	Required	Example	Type / notes

batchId	path	Yes	55555555-5555-5555-5555-555555555555	string

Dummy success response

```
{
  "data": [
    {
      "id": "11111111-1111-1111-1111-111111111111",
      "organizationId": "22222222-2222-2222-2222-222222222222",
      "batchId": "55555555-5555-5555-5555-555555555555",
      "catalogItemId": "66666666-6666-6666-6666-666666666666",
      "ledger": "COMPANY",
      "category": "CHICKS",
      "expenseDate": "2026-05-12",
      "description": "Starter feed purchase",
      "quantity": 1,
      "unit": "kg",
      "rate": 1,
      "totalAmount": 12500,
      "vendorName": "Sample Name",
      "invoiceNumber": "INV-2026-001",
      "billPhotoUrl": "https://cdn.example.com/expenses/bill-01.jpg",
      "paymentStatus": "PENDING",
      "paidAmount": 45000,
      "approvalStatus": "APPROVED",
      "approvedById": "sample",
      "approvedAt": "sample",
      "rejectedReason": "sample",
      "notes": "Captured from mobile app",
      "clientReferenceId": "sample",
      "createdById": "sample",
      "createdAt": "2026-05-12T07:18:15.474Z",
      "updatedAt": "2026-05-12T07:18:15.474Z"
    }
  ]
}
```

POST /api/v1/batches/{batchId}/costs

Purpose: Backward-compatible alias of POST /batches/{batchId}/expenses for older clients.

Implement in UI: Batch detail > expenses (legacy alias)

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

`Sample request line: POST /api/v1/batches/55555555-5555-5555-5555-555555555555/costs`

Note: Use /expenses for all new frontend work.

Note: The same CreateBatchExpenseRequest body is accepted here.

Parameters

Name	In	Required	Example	Type / notes
batchId	path	Yes	55555555-5555-5555-5555-555555555555	string

Dummy request body

```
{
  "ledger": "COMPANY",
  "catalogItemId": "66666666-6666-6666-6666-666666666666",
  "category": "FEED",
  "expenseDate": "2026-05-12",
  "description": "Starter feed purchase",
```

```
  "quantity": 1200,
  "unit": "kg",
  "rate": 36.5,
  "vendorName": "Local Feed Supplier",
  "invoiceNumber": "INV-2026-001",
  "notes": "Loaded from tablet app",
  "clientReferenceId": "mobile-cost-0001"
}
```

Dummy success response

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "batchId": "55555555-5555-5555-5555-555555555555",
  "catalogItemId": "66666666-6666-6666-6666-666666666666",
  "ledger": "COMPANY",
  "category": "CHICKS",
  "expenseDate": "2026-05-12",
  "description": "Starter feed purchase",
  "quantity": 1,
  "unit": "kg",
  "rate": 1,
  "totalAmount": 12500,
  "vendorName": "Sample Name",
  "invoiceNumber": "INV-2026-001",
  "billPhotoUrl": "https://cdn.example.com/expenses/bill-01.jpg",
  "paymentStatus": "PENDING",
  "paidAmount": 45000,
  "approvalStatus": "APPROVED",
  "approvedById": "sample",
  "approvedAt": "sample",
  "rejectedReason": "sample",
  "notes": "Captured from mobile app",
  "clientReferenceId": "sample",
  "createdById": "sample",
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```

GET /api/v1/batches/{batchId}/daily-logs

Purpose: Show daily flock history.

Implement in UI: Batch detail > daily entry

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

`Sample request line: GET /api/v1/batches/55555555-5555-5555-5555-555555555555/daily-logs`

Note: Returned logs are ordered by logDate descending.

Note: Use one screen for create and edit to keep field parity.

Parameters

Name	In	Required	Example	Type / notes
batchId	path	Yes	55555555-5555-5555-5555-555555555555	string

Dummy success response

```
{
  "data": [
    {
      "id": "11111111-1111-1111-1111-111111111111",
      "organizationId": "22222222-2222-2222-2222-222222222222",
```

```
      "batchId": "55555555-5555-5555-5555-555555555555",
      "logDate": "2026-05-12",
      "openingBirdCount": 2480,
      "mortalityCount": 8,
      "cullCount": 3,
      "feedConsumedKg": 1480.5,
      "waterConsumedLtr": 3200,
      "avgWeightGrams": 2350,
      "notes": "Captured from mobile app",
      "clientReferenceId": "sample",
      "recordedById": "sample",
      "correctedById": "sample",
      "createdAt": "2026-05-12T07:18:15.474Z",
      "updatedAt": "2026-05-12T07:18:15.474Z"
    }
  ]
}
```


POST /api/v1/batches/{batchId}/daily-logs

Purpose: Create the daily flock record for mortality, feed, water, and average weight.

Implement in UI: Batch detail > daily entry

Auth required: Yes

Allowed roles: OWNER, SUPERVISOR, FARMER

`Sample request line: POST /api/v1/batches/55555555-5555-5555-5555-555555555555/daily-logs`

Note: Farmers, supervisors, and owners can all add daily logs.

Note: clientReferenceId supports offline capture reconciliation.

Parameters

Name	In	Required	Example	Type / notes
batchId	path	Yes	55555555-5555-5555-5555-555555555555	string

Dummy request body

```
{
  "logDate": "2026-05-12",
  "openingBirdCount": 4992,
  "mortalityCount": 8,
  "cullCount": 3,
  "feedConsumedKg": 1480.5,
  "waterConsumedLtr": 3200,
  "avgWeightGrams": 1780,
  "notes": "Hot afternoon, intake slightly lower.",
  "clientReferenceId": "mobile-log-0001"
}
```

Dummy success response


```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "batchId": "55555555-5555-5555-5555-555555555555",
  "logDate": "2026-05-12",
  "openingBirdCount": 2480,
  "mortalityCount": 8,
  "cullCount": 3,
  "feedConsumedKg": 1480.5,
  "waterConsumedLtr": 3200,
  "avgWeightGrams": 2350,
  "notes": "Captured from mobile app",
  "clientReferenceId": "sample",
```

```
  "recordedById": "sample",
  "correctedById": "sample",
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```

PUT /api/v1/batches/{batchId}/daily-logs/{dailyLogId}

Purpose: Update a previously captured daily log entry.

Implement in UI: Batch detail > daily entry correction

Auth required: Yes

Allowed roles: OWNER, SUPERVISOR, FARMER

`Sample request line: PUT /api/v1/batches/55555555-5555-5555-5555-555555555555/daily-logs/88888888-8888-8888-8888-888888888888`

Note: When OWNER or SUPERVISOR updates a log, correctedById is set automatically.

Note: Use this for review/correction flow rather than creating duplicate daily logs.

Parameters

Name	In	Required	Example	Type / notes
batchId	path	Yes	55555555-5555-5555-5555-555555555555	string
dailyLogId	path	Yes	88888888-8888-8888-8888-888888888888	string

Dummy request body

```
{
  "mortalityCount": 9,
  "cullCount": 3,
  "feedConsumedKg": 1492,
  "notes": "Supervisor corrected mortality count."
}
```

Dummy success response

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "batchId": "55555555-5555-5555-5555-555555555555",
  "logDate": "2026-05-12",
  "openingBirdCount": 2480,
  "mortalityCount": 8,
  "cullCount": 3,
  "feedConsumedKg": 1480.5,
  "waterConsumedLtr": 3200,
  "avgWeightGrams": 2350,
  "notes": "Captured from mobile app",
  "clientReferenceId": "sample",
  "recordedById": "sample",
  "correctedById": "sample",
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```

GET /api/v1/batches/{batchId}/expenses

Purpose: Show company and farmer expense history for the batch.

Implement in UI: Batch detail > expenses

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

`Sample request line: GET /api/v1/batches/55555555-5555-5555-5555-555555555555/expenses?ledger=sample`

Note: Use the optional ledger query parameter to build Company and Farmer tabs.

Note: billPhotoUrl and invoice fields support finance audit and attachment previews.

Parameters

Name	In	Required	Example	Type / notes
batchId	path	Yes	55555555-5555-5555-5555-555555555555	string
ledger	query	No	sample	string

Dummy success response

```
{
  "data": [
    {
      "id": "11111111-1111-1111-1111-111111111111",
      "organizationId": "22222222-2222-2222-2222-222222222222",
      "batchId": "55555555-5555-5555-5555-555555555555",
      "catalogItemId": "66666666-6666-6666-6666-666666666666",
      "ledger": "COMPANY",
      "category": "CHICKS",
      "expenseDate": "2026-05-12",
      "description": "Starter feed purchase",
      "quantity": 1,
      "unit": "kg",
      "rate": 1,
      "totalAmount": 12500,
      "vendorName": "Sample Name",
      "invoiceNumber": "INV-2026-001",
      "billPhotoUrl": "https://cdn.example.com/expenses/bill-01.jpg",
      "paymentStatus": "PENDING",
      "paidAmount": 45000,
      "approvalStatus": "APPROVED",
      "approvedById": "sample",
      "approvedAt": "sample",
      "rejectedReason": "sample",
      "notes": "Captured from mobile app",
      "clientReferenceId": "sample",
      "createdById": "sample",
      "createdAt": "2026-05-12T07:18:15.474Z",
      "updatedAt": "2026-05-12T07:18:15.474Z"
    }
  ]
}
```

POST /api/v1/batches/{batchId}/expenses

Purpose: Add a company-ledger or farmer-ledger expense entry for the batch.

Implement in UI: Batch detail > expenses

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

`Sample request line: POST /api/v1/batches/55555555-5555-5555-5555-555555555555/expenses`

Note: Pass ledger as COMPANY or FARMER so the backend posts the expense to the correct ledger.

Note: Provide totalAmount directly or let the backend compute it from quantity x rate.

Parameters

Name	In	Required	Example	Type / notes
batchId	path	Yes	55555555-5555-5555-5555-555555555555	string

Dummy request body

```
{
  "ledger": "COMPANY",
  "catalogItemId": "66666666-6666-6666-6666-666666666666",
  "category": "FEED",
  "expenseDate": "2026-05-12",
  "description": "Starter feed purchase",
  "quantity": 1200,
  "unit": "kg",
  "rate": 36.5,
  "vendorName": "Local Feed Supplier",
  "invoiceNumber": "INV-2026-001",
  "billPhotoUrl": "https://cdn.example.com/expenses/bill-01.jpg",
  "notes": "Loaded from tablet app",
  "clientReferenceId": "mobile-expense-0001"
}
```

Dummy success response

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "batchId": "55555555-5555-5555-5555-555555555555",
  "catalogItemId": "66666666-6666-6666-6666-666666666666",
  "ledger": "COMPANY",
  "category": "CHICKS",
  "expenseDate": "2026-05-12",
  "description": "Starter feed purchase",
  "quantity": 1,
  "unit": "kg",
  "rate": 1,
  "totalAmount": 12500,
  "vendorName": "Sample Name",
  "invoiceNumber": "INV-2026-001",
  "billPhotoUrl": "https://cdn.example.com/expenses/bill-01.jpg",
  "paymentStatus": "PENDING",
  "paidAmount": 45000,
  "approvalStatus": "APPROVED",
  "approvedById": "sample",
  "approvedAt": "sample",
  "rejectedReason": "sample",
  "notes": "Captured from mobile app",
  "clientReferenceId": "sample",
  "createdById": "sample",
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```

PUT /api/v1/batches/{batchId}/expenses/{expenseId}

Purpose: Add or edit an expense row against the selected ledger.

Implement in UI: Batch detail > expenses

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

`Sample request line: PUT /api/v1/batches/55555555-5555-5555-5555-555555555555/expenses/dddddddd-dddd-dddd-dddd-dddddddddddd`

Note: Use the optional ledger query parameter to build Company and Farmer tabs.

Note: billPhotoUrl and invoice fields support finance audit and attachment previews.

Parameters

Name	In	Required	Example	Type / notes
batchId	path	Yes	55555555-5555-5555-5555-555555555555	string
expenseId	path	Yes	dddddddd-dddd-dddd-dddd-dddddddddddd	string

Dummy request body

```
{
  "quantity": 1250,
  "rate": 36.5,
  "totalAmount": 45625,
  "paymentStatus": "PARTIAL",
  "notes": "Updated after invoice reconciliation."
}
```

Dummy success response

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "batchId": "55555555-5555-5555-5555-555555555555",
  "catalogItemId": "66666666-6666-6666-6666-666666666666",
  "ledger": "COMPANY",
  "category": "CHICKS",
  "expenseDate": "2026-05-12",
  "description": "Starter feed purchase",
  "quantity": 1,
  "unit": "kg",
  "rate": 1,
  "totalAmount": 12500,
  "vendorName": "Sample Name",
  "invoiceNumber": "INV-2026-001",
  "billPhotoUrl": "https://cdn.example.com/expenses/bill-01.jpg",
  "paymentStatus": "PENDING",
  "paidAmount": 45000,
  "approvalStatus": "APPROVED",
  "approvedById": "sample",
  "approvedAt": "sample",
  "rejectedReason": "sample",
  "notes": "Captured from mobile app",
  "clientReferenceId": "sample",
  "createdById": "sample",
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```

PATCH /api/v1/batches/{batchId}/expenses/{expenseId}/approval

Purpose: Approve or reject a batch expense that is waiting for review.

Implement in UI: Batch detail > farmer expense approval

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR

`Sample request line: PATCH /api/v1/batches/55555555-5555-5555-5555-555555555555/expenses/dddddddd-dddd-dddd-dddd-dddddddddddd/approval`

Note: Use this for farmer-ledger expense approval queues when the organization requires review.

Note: If rejecting, send rejectedReason so the frontend can show actionable feedback.

Parameters

Name	In	Required	Example	Type / notes
batchId	path	Yes	55555555-5555-5555-5555-555555555555	string
expenseId	path	Yes	dddddddd-dddd-dddd-dddd-dddddddddddd	string

Dummy request body

```
{
  "approvalStatus": "APPROVED"
}
```

Dummy success response

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "batchId": "55555555-5555-5555-5555-555555555555",
  "catalogItemId": "66666666-6666-6666-6666-666666666666",
  "ledger": "COMPANY",
  "category": "CHICKS",
  "expenseDate": "2026-05-12",
  "description": "Starter feed purchase",
  "quantity": 1,
  "unit": "kg",
  "rate": 1,
  "totalAmount": 12500,
  "vendorName": "Sample Name",
  "invoiceNumber": "INV-2026-001",
  "billPhotoUrl": "https://cdn.example.com/expenses/bill-01.jpg",
  "paymentStatus": "PENDING",
  "paidAmount": 45000,
  "approvalStatus": "APPROVED",
  "approvedById": "sample",
  "approvedAt": "sample",
  "rejectedReason": "sample",
  "notes": "Captured from mobile app",
  "clientReferenceId": "sample",
  "createdById": "sample",
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```

GET /api/v1/batches/{batchId}/pnl

Purpose: Load separated company P&L and farmer earning views for the batch.

Implement in UI: Batch detail > P&L

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

`Sample request line: GET /api/v1/batches/55555555-5555-5555-5555-555555555555/pnl`

Note: Use this for final review screens and profitability tabs.

Note: Company and farmer numbers are intentionally separated because not every expense belongs to the same ledger.

Parameters

Name	In	Required	Example	Type / notes

batchId	path	Yes	55555555-5555-5555-5555-555555555555	string

Dummy success response

```
{
  "batchId": "55555555-5555-5555-5555-555555555555",
  "batchCode": "BATCH-MAY-2026-01",
  "company": {
    "netProfitOrLoss": 1,
    "expenses": 1,
    "salesRevenue": 1
  },
  "farmer": {
    "netEarnings": 1,
    "expenses": 1,
    "incentives": 1,
    "growingIncome": 1
  }
}
```

GET /api/v1/batches/{batchId}/sales

Purpose: List all sale transactions for the batch.

Implement in UI: Batch closure > sales

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

`Sample request line: GET /api/v1/batches/55555555-5555-5555-5555-555555555555/sales`

Note: Multiple sale rows are supported for split trader dispatches.

Note: Combine with settlement and P&L endpoints for closure screens.

Parameters

Name	In	Required	Example	Type / notes
batchId	path	Yes	55555555-5555-5555-5555-555555555555	string

Dummy success response

```
{
  "data": [
    {
      "id": "11111111-1111-1111-1111-111111111111",
      "organizationId": "22222222-2222-2222-2222-222222222222",
      "batchId": "55555555-5555-5555-5555-555555555555",
      "traderId": "77777777-7777-7777-7777-777777777777",
      "traderName": "Sample Name",
      "saleDate": "2026-05-12",
      "vehicleNumber": "MP09AB1234",
      "birdCount": 2480,
      "totalWeightKg": 2350,
      "averageWeightKg": 2350,
      "loadingMortalityCount": 8,
      "ratePerKg": 112,
      "grossAmount": 12500,
      "transportCharge": 1,
      "commissionCharge": 1,
      "otherDeduction": 750,
      "netAmount": 12500,
      "paymentReceivedAmount": 12500,
      "paymentStatus": "PENDING",
      "status": "CANCELLED",
```

```
      "notes": "Captured from mobile app",
      "clientReferenceId": "sample",
      "createdById": "sample",
      "finalizedById": "33333333-3333-3333-3333-333333333333",
      "finalizedAt": "2026-05-12T07:18:15.474Z",
      "createdAt": "2026-05-12T07:18:15.474Z",
      "updatedAt": "2026-05-12T07:18:15.474Z"
    }
  ]
}
```


POST /api/v1/batches/{batchId}/sales

Purpose: Create a trader sale entry for the batch, either as a draft or a fully priced sale row.

Implement in UI: Batch closure > sales

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

`Sample request line: POST /api/v1/batches/55555555-5555-5555-5555-555555555555/sales`

Note: For staged closing flows, create a DRAFT sale first and finalize commercial values later.

Note: If grossAmount is omitted, the backend calculates totalWeightKg x ratePerKg.

Parameters

Name	In	Required	Example	Type / notes
batchId	path	Yes	55555555-5555-5555-5555-555555555555	string

Dummy request body


```
{
  "traderId": "77777777-7777-7777-7777-777777777777",
  "saleDate": "2026-05-12",
  "vehicleNumber": "MP09AB1234",
  "birdCount": 2480,
  "totalWeightKg": 5320.75,
  "averageWeightKg": 2.15,
  "loadingMortalityCount": 4,
  "status": "DRAFT",
  "notes": "Dispatch completed, commercials pending confirmation.",
  "clientReferenceId": "mobile-sale-0001"
}
```


Dummy success response


```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "batchId": "55555555-5555-5555-5555-555555555555",
  "traderId": "77777777-7777-7777-7777-777777777777",
  "traderName": "Sample Name",
  "saleDate": "2026-05-12",
  "vehicleNumber": "MP09AB1234",
  "birdCount": 2480,
  "totalWeightKg": 2350,
  "averageWeightKg": 2350,
  "loadingMortalityCount": 8,
  "ratePerKg": 112,
  "grossAmount": 12500,
  "transportCharge": 1,
  "commissionCharge": 1,
  "otherDeduction": 750,
  "netAmount": 12500,
  "paymentReceivedAmount": 12500,
```

```
  "paymentStatus": "PENDING",
  "status": "CANCELLED",
  "notes": "Captured from mobile app",
  "clientReferenceId": "sample",
  "createdById": "sample",
  "finalizedById": "33333333-3333-3333-3333-333333333333",
  "finalizedAt": "2026-05-12T07:18:15.474Z",
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```

PATCH /api/v1/batches/{batchId}/sales/{saleId}/finalize

Purpose: Lock in final sale pricing, deductions, receipts, and status for a draft sale row.

Implement in UI: Batch closure > finalize sale

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS

`Sample request line: PATCH /api/v1/batches/55555555-5555-5555-5555-555555555555/sales/eeeeeeee-eeee-eeee-eeee-eeeeeeeeeeee/finalize`

Note: Use this when dispatch details are known before final commercial settlement is available.

Note: Only OWNER and ACCOUNTS users can finalize sale rows.

Parameters

Name	In	Required	Example	Type / notes
batchId	path	Yes	55555555-5555-5555-5555-555555555555	string
saleId	path	Yes	eeeeeeee-eeee-eeee-eeee-eeeeeeeeeeee	string

Dummy request body

```
{
  "ratePerKg": 112,
  "grossAmount": 595924,
  "transportCharge": 3500,
  "commissionCharge": 1500,
  "otherDeduction": 500,
  "netAmount": 590424,
  "paymentReceivedAmount": 450000,
  "paymentStatus": "PARTIAL",
  "notes": "Final trader statement received."
}
```

Dummy success response

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "batchId": "55555555-5555-5555-5555-555555555555",
  "traderId": "77777777-7777-7777-7777-777777777777",
  "traderName": "Sample Name",
  "saleDate": "2026-05-12",
  "vehicleNumber": "MP09AB1234",
  "birdCount": 2480,
  "totalWeightKg": 2350,
  "averageWeightKg": 2350,
  "loadingMortalityCount": 8,
  "ratePerKg": 112,
  "grossAmount": 12500,
  "transportCharge": 1,
  "commissionCharge": 1,
```

```
  "otherDeduction": 750,
  "netAmount": 12500,
  "paymentReceivedAmount": 12500,
  "paymentStatus": "PENDING",
  "status": "CANCELLED",
  "notes": "Captured from mobile app",
  "clientReferenceId": "sample",
  "createdById": "sample",
  "finalizedById": "33333333-3333-3333-3333-333333333333",
  "finalizedAt": "2026-05-12T07:18:15.474Z",
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```

GET /api/v1/batches/{batchId}/settlement

Purpose: Load the computed farmer settlement for the batch.

Implement in UI: Batch closure > settlement

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

`Sample request line: GET /api/v1/batches/55555555-5555-5555-5555-555555555555/settlement`

Note: Use this to render the final farmer payout card after sales and expenses are ready.

Note: The response includes gross growing charges, deductions, expense total, and net payable.

Parameters

Name	In	Required	Example	Type / notes
batchId	path	Yes	55555555-5555-5555-5555-555555555555	string

Dummy success response

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "batchId": "55555555-5555-5555-5555-555555555555",
  "farmerId": "33333333-3333-3333-3333-333333333333",
  "farmerName": "Ramesh Patel",
  "payoutRate": 21,
  "payoutUnit": "PER_KG_SOLD",
  "baseQuantity": 4950,
  "growingCharges": 103950,
  "performanceBonus": 4200,
  "incentiveAmount": 1800,
  "otherDeductions": 750,
  "farmerExpenseTotal": 6200,
  "netPayable": 99750,
  "paymentStatus": "PENDING",
  "status": "FINALIZED",
  "remarks": "Reviewed and aligned with current batch ledger.",
  "finalizedById": "33333333-3333-3333-3333-333333333333",
  "finalizedAt": "2026-05-12T07:18:15.474Z",
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```

POST /api/v1/batches/{batchId}/settlement

Purpose: Create or update the farmer settlement using payout rules, incentives, and deductions.

Implement in UI: Batch closure > settlement

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS

`Sample request line: POST /api/v1/batches/55555555-5555-5555-5555-555555555555/settlement`

Note: The backend computes base quantity and growing charges from batch and payout rules.

Note: Use this once the batch is in settlement-ready stage.

Parameters

Name	In	Required	Example	Type / notes
batchId	path	Yes	55555555-5555-5555-5555-555555555555	string

Dummy request body

```
{
  "payoutRate": 21,
  "payoutUnit": "PER_KG_SOLD",
  "performanceBonus": 4200,
  "incentiveAmount": 1800,
  "otherDeductions": 750,
  "paymentStatus": "PENDING",
  "remarks": "Prepared for farmer approval."
}
```

Dummy success response

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "batchId": "55555555-5555-5555-5555-555555555555",
  "farmerId": "33333333-3333-3333-3333-333333333333",
  "farmerName": "Ramesh Patel",
  "payoutRate": 21,
  "payoutUnit": "PER_KG_SOLD",
  "baseQuantity": 4950,
  "growingCharges": 103950,
  "performanceBonus": 4200,
  "incentiveAmount": 1800,
  "otherDeductions": 750,
  "farmerExpenseTotal": 6200,
  "netPayable": 99750,
  "paymentStatus": "PENDING",
  "status": "FINALIZED",
  "remarks": "Reviewed and aligned with current batch ledger.",
  "finalizedById": "33333333-3333-3333-3333-333333333333",
  "finalizedAt": "2026-05-12T07:18:15.474Z",
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```

PATCH /api/v1/batches/{batchId}/status

Purpose: Create, edit, or change the lifecycle status of a batch.

Implement in UI: Batch list / batch detail / batch form

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR

`Sample request line: PATCH /api/v1/batches/55555555-5555-5555-5555-555555555555/status`

Note: Use status to drive lifecycle actions such as active work, sales running, settlement pending, and closed.

Note: Farmers only see batches for farms they can access.

Parameters

Name	In	Required	Example	Type / notes
batchId	path	Yes	55555555-5555-5555-5555-555555555555	string

Dummy request body

```
{
  "status": "SALES_RUNNING",
  "actualCloseDate": null
}
```

Dummy success response

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "farmId": "44444444-4444-4444-4444-444444444444",
  "farmName": "Shivam Farm",
  "code": "BATCH-MAY-2026-01",
  "placementDate": "2026-05-12",
  "placementCount": 5000,
  "totalChicksPurchased": 5050,
  "freeChicks": 50,
  "chargeableChicks": 5000,
  "placementMortality": 12,
  "chickCostTotal": 220000,
  "chickRatePerBird": 44,
  "ratePerChick": 1,
  "chickTransportCharge": 5500,
  "sourceHatchery": "Sunrise Hatchery",
  "vendorName": "Sample Name",
  "targetCloseDate": "2026-05-12",
  "actualCloseDate": "2026-05-12",
  "status": "ACTIVE",
  "lockedAt": "2026-05-12T07:18:15.474Z",
  "notes": "Captured from mobile app",
  "summary": {
    "currentAgeDays": 31,
    "liveBirds": 4876,
    "mortalityCount": 8,
    "cullCount": 3,
    "mortalityPercent": 1,
    "soldBirds": 2480,
    "totalFeedConsumedKg": 1480.5,
    "totalWeightSoldKg": 2350,
    "averageWeightGrams": 2350,
    "fcr": 1.68
  },
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```

GET /api/v1/batches/{batchId}/treatments

Purpose: Show treatment history.

Implement in UI: Batch detail > treatment / vaccine log

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

```
Sample request line: GET /api/v1/batches/55555555-5555-5555-5555-555555555555/treatments
```

Note: Returned treatments are ordered newest first.

Note: Useful for both farmer capture and supervisor review.

Parameters

Name	In	Required	Example	Type / notes
batchId	path	Yes	55555555-5555-5555-5555-555555555555	string

Dummy success response

```
{
  "data": [
    {
      "id": "11111111-1111-1111-1111-111111111111",
      "organizationId": "22222222-2222-2222-2222-222222222222",
      "batchId": "55555555-5555-5555-5555-555555555555",
      "dailyLogId": "88888888-8888-8888-8888-888888888888",
      "treatmentDate": "2026-05-12",
      "kind": "VACCINATION",
      "catalogItemId": "66666666-6666-6666-6666-666666666666",
      "treatmentName": "Sample Name",
      "dosage": "1 ml / 10 birds",
      "birdCount": 2480,
      "notes": "Captured from mobile app",
      "clientReferenceId": "sample",
      "administeredById": "sample",
      "createdAt": "2026-05-12T07:18:15.474Z",
      "updatedAt": "2026-05-12T07:18:15.474Z"
    }
  ]
}
```

POST /api/v1/batches/{batchId}/treatments

Purpose: Record treatment, medication, or vaccination actions for the batch.

Implement in UI: Batch detail > treatment / vaccine log

Auth required: Yes

Allowed roles: OWNER, SUPERVISOR, FARMER

`Sample request line: POST /api/v1/batches/55555555-5555-5555-5555-555555555555/treatments`

Note: dailyLogId is optional but should be supplied when the treatment belongs to a specific daily log.

Note: clientReferenceId supports offline replay handling.

Parameters

Name	In	Required	Example	Type / notes
batchId	path	Yes	55555555-5555-5555-5555-555555555555	string

Dummy request body

```
{
  "dailyLogId": "88888888-8888-8888-8888-888888888888",
  "treatmentDate": "2026-05-12",
  "kind": "VACCINATION",
  "catalogItemId": "66666666-6666-6666-6666-666666666666",
  "treatmentName": "Newcastle Vaccine",
  "dosage": "1 ml / 10 birds",
  "birdCount": 5000,
  "notes": "Routine vaccination",
```

```
  "clientReferenceId": "mobile-treatment-0001"
}
```

Dummy success response

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "batchId": "55555555-5555-5555-5555-555555555555",
  "dailyLogId": "88888888-8888-8888-8888-888888888888",
  "treatmentDate": "2026-05-12",
  "kind": "VACCINATION",
  "catalogItemId": "66666666-6666-6666-6666-666666666666",
  "treatmentName": "Sample Name",
  "dosage": "1 ml / 10 birds",
  "birdCount": 2480,
  "notes": "Captured from mobile app",
  "clientReferenceId": "sample",
  "administeredById": "sample",
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```

5.7 Inventory

POST /api/v1/inventory/allocate

Purpose: Allocate central stock to a specific batch and create the corresponding ledger movement.

Implement in UI: Inventory > allocate stock to batch

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR

`Sample request line: POST /api/v1/inventory/allocate`

Note: Use this after purchases are stocked in and material is issued to a batch.

Note: The backend validates item and batch ownership before posting the movement.

Dummy request body

```
{
  "batchId": "55555555-5555-5555-5555-555555555555",
  "catalogItemId": "66666666-6666-6666-6666-666666666666",
  "quantity": 400,
  "remarks": "Released to Batch MAY-2026-01 feed store."
}
```

Dummy success response

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "catalogItemId": "66666666-6666-6666-6666-666666666666",
  "catalogItemName": "Starter Feed",
  "batchId": "55555555-5555-5555-5555-555555555555",
  "movementType": "PURCHASE",
  "movementDate": "sample",
  "quantityIn": 2500,
  "quantityOut": 400,
  "balanceAfter": 2100,
  "referenceType": "PURCHASE",
  "referenceId": "sample",
  "notes": "Captured from mobile app",
  "createdById": "sample",
  "createdAt": "2026-05-12T07:18:15.474Z"
}
```

GET /api/v1/inventory/ledger

Purpose: Load stock movement history with optional filtering by catalog item or batch.

Implement in UI: Inventory > stock ledger

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

```
Sample request line: GET /api/v1/inventory/ledger?catalogItemId=66666666-6666-6666-6666-
666666666666&batchId=55555555-5555-5555-5555-555555555555
```

Note: Use catalogItemId for item-wise stock history and batchId for batch allocation traceability.

Note: This powers stock balance, inward, outward, and audit views.

Parameters

Name	In	Required	Example	Type / notes
catalogItemId	query	No	66666666-6666-6666-6666-666666666666	string
batchId	query	No	55555555-5555-5555-5555-555555555555	string

Dummy success response

```
{
  "data": [
    {
      "id": "11111111-1111-1111-1111-111111111111",
      "organizationId": "22222222-2222-2222-2222-222222222222",
      "catalogItemId": "66666666-6666-6666-6666-666666666666",
      "catalogItemName": "Starter Feed",
      "batchId": "55555555-5555-5555-5555-555555555555",
      "movementType": "PURCHASE",
      "movementDate": "sample",
      "quantityIn": 2500,
      "quantityOut": 400,
      "balanceAfter": 2100,
      "referenceType": "PURCHASE",
      "referenceId": "sample",
      "notes": "Captured from mobile app",
      "createdById": "sample",
      "createdAt": "2026-05-12T07:18:15.474Z"
    }
  ]
}
```

5.8 Finance

GET /api/v1/finance/entries

Purpose: Manage organization-level finance ledger entries.

Implement in UI: Finance

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS

```
Sample request line: GET /api/v1/finance/entries?page=1&limit=20&search=shivam
```

Note: Finance endpoints power purchase, payment, investment, and non-batch ledger flows.

Note: Use paymentStatus fields to drive pending, partial, and paid badges across finance screens.

Parameters

Name	In	Required	Example	Type / notes
page	query	No	1	number
limit	query	No	20	number
search	query	No	shivam	string

Dummy success response

```
{
  "data": [
    {
      "id": "11111111-1111-1111-1111-111111111111",
      "organizationId": "22222222-2222-2222-2222-222222222222",
      "type": "INVESTMENT",
      "amount": 12500,
      "paymentStatus": "PENDING",
      "entryDate": "sample",
      "description": "Starter feed purchase",
      "notes": "Captured from mobile app",
      "createdById": "sample",
      "createdAt": "2026-05-12T07:18:15.474Z",
      "updatedAt": "2026-05-12T07:18:15.474Z"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 1,
    "totalPages": 1
  }
}
```

POST /api/v1/finance/entries

Purpose: Post owner/accounts-level investment, other income, or other expense entries not tied to a batch transaction.

Implement in UI: Finance > organization ledger

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS

`Sample request line: POST /api/v1/finance/entries`

Note: Use this for capital infusion and non-batch ledger adjustments.

Note: These values feed the financial dashboard and overview reports.

Dummy request body

```
{
  "type": "INVESTMENT",
  "amount": 250000,
  "paymentStatus": "PAID",
  "entryDate": "2026-05-12",
  "description": "Owner capital injection for working capital.",
  "notes": "Transferred from promoter account."
}
```

Dummy success response

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "type": "INVESTMENT",
```

```
  "amount": 12500,
  "paymentStatus": "PENDING",
  "entryDate": "sample",
  "description": "Starter feed purchase",
  "notes": "Captured from mobile app",
  "createdById": "sample",
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```

GET /api/v1/finance/payments

Purpose: Manage payment in or out entries and reference-linked settlement.

Implement in UI: Finance

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR

`Sample request line: GET /api/v1/finance/payments?page=1&limit=20&search=shivam`

Note: Finance endpoints power purchase, payment, investment, and non-batch ledger flows.

Note: Use paymentStatus fields to drive pending, partial, and paid badges across finance screens.

Parameters

Name	In	Required	Example	Type / notes
page	query	No	1	number
limit	query	No	20	number
search	query	No	shivam	string

Dummy success response

```
{
  "data": [
    {
      "id": "11111111-1111-1111-1111-111111111111",
      "organizationId": "22222222-2222-2222-2222-222222222222",
      "batchId": "55555555-5555-5555-5555-555555555555",
      "partyName": "Raghav Suppliers",
      "paymentType": "PURCHASE",
      "direction": "INBOUND",
      "amount": 12500,
      "paymentDate": "sample",
      "referenceType": "PURCHASE",
      "referenceId": "sample",
      "notes": "Captured from mobile app",
      "createdById": "sample",
      "createdAt": "2026-05-12T07:18:15.474Z",
      "updatedAt": "2026-05-12T07:18:15.474Z"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 1,
    "totalPages": 1
  }
}
```

POST /api/v1/finance/payments

Purpose: Record a payment movement against a purchase, expense, sale receipt, settlement, or general party transaction.

Implement in UI: Finance > payment register

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR

`Sample request line: POST /api/v1/finance/payments`

Note: referenceType and referenceId let the backend update related payment status automatically.

Note: Use OUTBOUND for payouts or expenses and INBOUND for sale receipts or recoveries.

Dummy request body

```
{
  "batchId": "55555555-5555-5555-5555-555555555555",
  "partyName": "Local Feed Supplier",
  "paymentType": "PURCHASE",
  "direction": "OUTBOUND",
  "amount": 45000,
  "paymentDate": "2026-05-12",
  "referenceType": "PURCHASE",
  "referenceId": "ffffffff-ffff-ffff-ffff-ffffffffffff",
  "notes": "First installment released."
}
```

Dummy success response

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "batchId": "55555555-5555-5555-5555-555555555555",
  "partyName": "Raghav Suppliers",
  "paymentType": "PURCHASE",
  "direction": "INBOUND",
  "amount": 12500,
  "paymentDate": "sample",
  "referenceType": "PURCHASE",
  "referenceId": "sample",
  "notes": "Captured from mobile app",
  "createdById": "sample",
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```

GET /api/v1/finance/purchases

Purpose: Manage purchase records and payable state.

Implement in UI: Finance

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR

`Sample request line: GET /api/v1/finance/purchases?page=1&limit=20&search=shivam`

Note: Finance endpoints power purchase, payment, investment, and non-batch ledger flows.

Note: Use paymentStatus fields to drive pending, partial, and paid badges across finance screens.

Parameters

Name	In	Required	Example	Type / notes
page	query	No	1	number

limit	query	No	20	number
search	query	No	shivam	string

Dummy success response

```
{
  "data": [
    {
      "id": "11111111-1111-1111-1111-111111111111",
      "organizationId": "22222222-2222-2222-2222-222222222222",
      "batchId": "55555555-5555-5555-5555-555555555555",
      "purchaseType": "FEED",
      "vendorName": "Sample Name",
      "catalogItemId": "66666666-6666-6666-6666-666666666666",
      "itemName": "Starter Feed",
      "quantity": 1,
      "unit": "kg",
      "unitCost": 1,
      "totalAmount": 12500,
      "invoiceNumber": "INV-2026-001",
      "paymentStatus": "PENDING",
      "paidAmount": 45000,
      "purchaseDate": "sample",
      "attachmentUrl": "https://cdn.example.com/purchases/invoice-01.jpg",
      "remarks": "Reviewed and aligned with current batch ledger.",
      "clientReferenceId": "sample",
      "createdById": "sample",
      "createdAt": "2026-05-12T07:18:15.474Z",
      "updatedAt": "2026-05-12T07:18:15.474Z"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 1,
    "totalPages": 1
  }
}
```

POST /api/v1/finance/purchases

Purpose: Create a purchase record that can optionally be tied to a batch and catalog item.

Implement in UI: Finance > purchase entry

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR

`Sample request line: POST /api/v1/finance/purchases`

Note: Use this for feed, medicine, vaccine, chick, and equipment procurement.

Note: Attachment or invoice URLs can be stored for finance audit trails.

Dummy request body

```
{
  "batchId": "55555555-5555-5555-5555-555555555555",
  "purchaseType": "FEED",
  "vendorName": "Local Feed Supplier",
  "catalogItemId": "66666666-6666-6666-6666-666666666666",
  "itemName": "Starter Feed",
  "quantity": 2500,
  "unit": "kg",
  "unitCost": 36.5,
  "totalAmount": 91250,
  "invoiceNumber": "INV-2026-101",
```

```
  "paymentStatus": "PENDING",
  "purchaseDate": "2026-05-12",
  "attachmentUrl": "https://cdn.example.com/purchases/invoice-01.jpg",
  "remarks": "Initial purchase for new batch.",
  "clientReferenceId": "purchase-0001"
}
```

Dummy success response

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "batchId": "55555555-5555-5555-5555-555555555555",
  "purchaseType": "FEED",
  "vendorName": "Sample Name",
  "catalogItemId": "66666666-6666-6666-6666-666666666666",
  "itemName": "Starter Feed",
  "quantity": 1,
  "unit": "kg",
  "unitCost": 1,
  "totalAmount": 12500,
  "invoiceNumber": "INV-2026-001",
  "paymentStatus": "PENDING",
  "paidAmount": 45000,
  "purchaseDate": "sample",
  "attachmentUrl": "https://cdn.example.com/purchases/invoice-01.jpg",
  "remarks": "Reviewed and aligned with current batch ledger.",
  "clientReferenceId": "sample",
  "createdById": "sample",
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```

PUT /api/v1/finance/purchases/{purchaseId}

Purpose: Update a previously recorded purchase entry, including payment status and invoice details.

Implement in UI: Finance > purchase edit

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS

`Sample request line: PUT /api/v1/finance/purchases/ffffffff-ffff-ffff-ffff-ffffffffffff`

Note: Restricted to OWNER and ACCOUNTS roles.

Note: Use this when invoice totals or payment state change after initial entry.

Parameters

Name	In	Required	Example	Type / notes
purchaseId	path	Yes	ffffffff-ffff-ffff-ffff-ffffffffffff	string

Dummy request body

```
{
  "totalAmount": 90250,
  "paymentStatus": "PARTIAL",
  "remarks": "Vendor discount applied after final invoice."
}
```

Dummy success response

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "batchId": "55555555-5555-5555-5555-555555555555",
  "purchaseType": "FEED",
  "vendorName": "Sample Name",
```

```
  "catalogItemId": "66666666-6666-6666-6666-666666666666",
  "itemName": "Starter Feed",
  "quantity": 1,
  "unit": "kg",
  "unitCost": 1,
  "totalAmount": 12500,
  "invoiceNumber": "INV-2026-001",
  "paymentStatus": "PENDING",
  "paidAmount": 45000,
  "purchaseDate": "sample",
  "attachmentUrl": "https://cdn.example.com/purchases/invoice-01.jpg",
  "remarks": "Reviewed and aligned with current batch ledger.",
  "clientReferenceId": "sample",
  "createdById": "sample",
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```

5.9 Notifications

GET /api/v1/notifications

Purpose: Load alerts, reminders, and workflow notifications for the signed-in user.

Implement in UI: Notification center

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

`Sample request line: GET /api/v1/notifications?unreadOnly=true`

Note: Use unreadOnly=true for badge counts or focused alert drawers.

Note: Notification payloads can include farmId, batchId, targetType, and targetId for deep-linking.

Parameters

Name	In	Required	Example	Type / notes
unreadOnly	query	No	true	boolean

Dummy success response

```
{
  "data": [
    {
      "id": "11111111-1111-1111-1111-111111111111",
      "organizationId": "22222222-2222-2222-2222-222222222222",
      "farmId": "44444444-4444-4444-4444-444444444444",
      "batchId": "55555555-5555-5555-5555-555555555555",
      "type": "FEED_ALERT",
      "severity": "WARNING",
      "title": "Feed stock running low",
      "message": "Starter Feed stock is below the configured reorder level.",
      "targetType": "sample",
      "targetId": "bbbbbbbb-bbbb-bbbb-bbbb-bbbbbbbbbbbb",
      "isRead": true,
      "readAt": "sample",
      "readById": "sample",
      "createdAt": "2026-05-12T07:18:15.474Z"
    }
  ]
}
```

PATCH /api/v1/notifications/{notificationId}/read

Purpose: Mark a notification as read for the current user.

Implement in UI: Notification center > mark read

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

`Sample request line: PATCH /api/v1/notifications/12121212-1212-1212-1212-121212121212/read`

Note: Use this after opening the linked detail view or dismissing an alert card.

Note: The response returns the updated read state and timestamp.

Parameters

Name	In	Required	Example	Type / notes
notificationId	path	Yes	12121212-1212-1212-1212-121212121212	string

Dummy success response

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "farmId": "44444444-4444-4444-4444-444444444444",
  "batchId": "55555555-5555-5555-5555-555555555555",
  "type": "FEED_ALERT",
  "severity": "WARNING",
  "title": "Feed stock running low",
  "message": "Starter Feed stock is below the configured reorder level.",
  "targetType": "sample",
  "targetId": "bbbbbbbb-bbbb-bbbb-bbbb-bbbbbbbbbbbb",
  "isRead": true,
  "readAt": "sample",
  "readById": "sample",
  "createdAt": "2026-05-12T07:18:15.474Z"
}
```

5.10 Reports

GET /api/v1/reports/batches/{batchId}/export/excel

Purpose: Download a ready-to-share Excel workbook for the selected batch.

Implement in UI: Batch report > share/export

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR

`Sample request line: GET /api/v1/reports/batches/55555555-5555-5555-5555-555555555555/export/excel`

Note: Treat this as a file download and request it as blob or binary on the frontend.

Note: The runtime response is a downloadable file even though Swagger shows no JSON response body.

Parameters

Name	In	Required	Example	Type / notes
batchId	path	Yes	55555555-5555-5555-5555-555555555555	string

Dummy success response

```
{
  "contentType": "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
  "disposition": "attachment; filename=\"BATCH-MAY-2026-01-report.xlsx\"",
```

```
  "body": "<binary .xlsx file>"
}
```

GET /api/v1/reports/batches/{batchId}/export/pdf

Purpose: Download a PDF summary for the selected batch.

Implement in UI: Batch report > share/export

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR

`Sample request line: GET /api/v1/reports/batches/55555555-5555-5555-5555-555555555555/export/pdf`

Note: Treat this as a file download and request it as blob or binary on the frontend.

Note: The runtime response is a downloadable file even though Swagger shows no JSON response body.

Parameters

Name	In	Required	Example	Type / notes
batchId	path	Yes	55555555-5555-5555-5555-555555555555	string

Dummy success response

```
{
  "contentType": "application/pdf",
  "disposition": "attachment; filename=\"BATCH-MAY-2026-01-report.pdf\"",
  "body": "<binary pdf file>"
}
```

GET /api/v1/reports/batches/{batchId}/summary

Purpose: Load performance and financial summaries for a batch or farm.

Implement in UI: Reports / exports

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

`Sample request line: GET /api/v1/reports/batches/55555555-5555-5555-5555-555555555555/summary`

Note: Use these APIs to build KPI cards, finance tables, and export actions.

Note: Batch and farm summaries already separate company expenses from farmer expenses.

Parameters

Name	In	Required	Example	Type / notes
batchId	path	Yes	55555555-5555-5555-5555-555555555555	string

Dummy success response

```
{
  "batchId": "55555555-5555-5555-5555-555555555555",
  "batchCode": "BATCH-MAY-2026-01",
  "farmId": "44444444-4444-4444-4444-444444444444",
  "farmName": "Shivam Farm",
  "status": "ACTIVE",
  "placementCount": 5000,
  "currentAgeDays": 31,
  "liveBirds": 4876,
  "mortalityCount": 8,
```

```
  "cullCount": 3,
  "soldBirdCount": 2480,
  "mortalityRate": 2.2,
  "totalFeedConsumedKg": 1480.5,
  "totalWeightSoldKg": 2350,
  "averageWeightGrams": 2350,
  "fcr": 1.68,
  "totalCompanyExpenses": 1,
  "totalFarmerExpenses": 1,
  "totalSales": 1,
  "companyProfitOrLoss": 1,
  "farmerGrowingIncome": 1,
  "farmerNetEarnings": 1,
  "settlementStatus": "sample"
}
```

GET /api/v1/reports/expenses

Purpose: Load batch-wise expense rows across company and farmer ledgers.

Implement in UI: Reports > expense register

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

`Sample request line: GET /api/v1/reports/expenses`

Note: Use this for finance audit tables and export-ready expense reporting.

Note: Each row already includes farm, batch, category, ledger, and amount context.

Dummy success response

```
[
  {
    "expenseId": "dddddddd-dddd-dddd-dddd-dddddddddddd",
    "batchId": "55555555-5555-5555-5555-555555555555",
    "batchCode": "BATCH-MAY-2026-01",
    "farmId": "44444444-4444-4444-4444-444444444444",
    "farmName": "Shivam Farm",
    "ledger": "COMPANY",
    "category": "CHICKS",
    "expenseDate": "2026-05-12",
    "description": "Starter feed purchase",
    "totalAmount": 12500
  }
]
```

GET /api/v1/reports/farms/{farmId}/summary

Purpose: Load performance and financial summaries for a batch or farm.

Implement in UI: Reports / exports

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

`Sample request line: GET /api/v1/reports/farms/44444444-4444-4444-4444-444444444444/summary`

Note: Use these APIs to build KPI cards, finance tables, and export actions.

Note: Batch and farm summaries already separate company expenses from farmer expenses.

Parameters

Name	In	Required	Example	Type / notes

farmId	path	Yes	44444444-4444-4444-4444-444444444444	string

Dummy success response

```
{
  "farmId": "44444444-4444-4444-4444-444444444444",
  "farmName": "Shivam Farm",
  "totalBatches": 1,
  "activeBatches": 1,
  "closedBatches": 1,
  "totalPlacementCount": 5000,
  "totalSales": 1,
  "totalCompanyExpenses": 1,
  "totalFarmerExpenses": 1,
  "companyProfitOrLoss": 1,
  "farmerNetEarnings": 1,
  "averageFcr": 1.68
}
```

GET /api/v1/reports/inventory

Purpose: Load current stock position, reorder levels, and low-stock flags by item.

Implement in UI: Reports > inventory snapshot

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR

`Sample request line: GET /api/v1/reports/inventory`

Note: Use this for stock summary widgets and procurement planning tables.

Note: This report is intended for OWNER, ACCOUNTS, and SUPERVISOR users.

Dummy success response

```
[
  {
    "itemId": "cccccccc-cccc-cccc-cccc-cccccccccccc",
    "itemName": "Starter Feed",
    "itemType": "sample",
    "currentStock": 1380,
    "reorderLevel": 500,
    "lowStock": true
  }
]
```

GET /api/v1/reports/overview

Purpose: Load consolidated operational, expense, inventory, settlement, or profitability report data.

Implement in UI: Reports / exports

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR

`Sample request line: GET /api/v1/reports/overview`

Note: Use these APIs to build KPI cards, finance tables, and export actions.

Note: Batch and farm summaries already separate company expenses from farmer expenses.

Dummy success response

```
{
  "totalFarms": 1,
```

```
  "totalBatches": 1,
  "activeBatches": 1,
  "closedBatches": 1,
  "totalUsers": 1,
  "liveBirds": 4876,
  "mortalityToday": 1,
  "pendingEntries": 2,
  "totalSales": 1,
  "totalCompanyExpenses": 1,
  "totalFarmerExpenses": 1,
  "companyProfitOrLoss": 1,
  "farmerNetEarnings": 1,
  "investmentTotal": 1250000,
  "pendingPayments": 1,
  "unreadNotifications": 1
}
```

GET /api/v1/reports/profitability

Purpose: Load batch-wise company profit/loss and farmer net earnings rows.

Implement in UI: Reports > profitability

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR

`Sample request line: GET /api/v1/reports/profitability`

Note: Use this for management review tables and summary charts.

Note: This is an owner/accounts/supervisor financial visibility endpoint.

Dummy success response

```
[
  {
    "batchId": "55555555-5555-5555-5555-555555555555",
    "batchCode": "BATCH-MAY-2026-01",
    "farmName": "Shivam Farm",
    "companyProfitOrLoss": 1,
    "farmerNetEarnings": 1
  }
]
```

GET /api/v1/reports/settlements

Purpose: Load farmer settlement rows with gross income, deductions, and status.

Implement in UI: Reports > settlements

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS, SUPERVISOR, FARMER

`Sample request line: GET /api/v1/reports/settlements`

Note: Use this for settlement tracking and farmer payout dashboards.

Note: Pairs well with batch settlement detail screens.

Dummy success response

```
[
  {
    "settlementId": "13131313-1313-1313-1313-131313131313",
    "batchId": "55555555-5555-5555-5555-555555555555",
    "batchCode": "BATCH-MAY-2026-01",
    "farmName": "Shivam Farm",
```

```
    "farmerName": "Ramesh Patel",
    "growingCharges": 103950,
    "incentives": 1,
    "farmerExpenses": 1,
    "deductions": 1,
    "netPayable": 99750,
    "status": "FINALIZED"
  }
]
```

5.11 Settings

GET /api/v1/settings/organization

Purpose: Load currency, alert thresholds, payout defaults, expense category rules, and finance approval settings.

Implement in UI: Settings > organization

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS

`Sample request line: GET /api/v1/settings/organization`

Note: This endpoint drives admin-side configuration screens.

Note: Frontend should read these values instead of assuming static payout or approval rules.

Dummy success response

```
{
  "currency": "INR",
  "mobileFirst": true,
  "expenseCategories": {
    "farmer": [
      "sample"
    ],
    "company": [
      "sample"
    ]
  },
  "payoutRules": {
    "defaultPayoutRate": 21,
    "defaultPayoutUnit": "PER_BIRD_PLACED"
  },
  "alertThresholds": {
    "pendingEntryDays": 1,
    "fcr": 1.68,
    "mortalityPercent": 1
  },
  "financialConfig": {
    "supervisorCanAddFarmerExpense": true,
    "supervisorCanAddCompanyExpense": true,
    "farmerExpenseRequiresApproval": true
  }
}
```

PUT /api/v1/settings/organization

Purpose: Update organization-wide payout, alert, expense, and financial-control settings.

Implement in UI: Settings > organization

Auth required: Yes

Allowed roles: OWNER, ACCOUNTS

`Sample request line: PUT /api/v1/settings/organization`

Note: Use partial payloads to update only the settings block being edited.

Note: Changes here affect settlement defaults and farmer-expense approval flow.

Dummy request body

```
{
  "payoutRules": {
    "defaultPayoutUnit": "PER_KG_SOLD",
    "defaultPayoutRate": 21
  },
  "alertThresholds": {
    "mortalityPercent": 3,
    "fcr": 1.85,
    "pendingEntryDays": 1
  },
  "financialConfig": {
    "farmerExpenseRequiresApproval": true,
    "supervisorCanAddCompanyExpense": true,
    "supervisorCanAddFarmerExpense": true
  }
}
```

Dummy success response

```
{
  "currency": "INR",
  "mobileFirst": true,
  "expenseCategories": {
    "farmer": [
      "sample"
    ],
    "company": [
      "sample"
    ]
  },
  "payoutRules": {
    "defaultPayoutRate": 21,
    "defaultPayoutUnit": "PER_BIRD_PLACED"
  },
  "alertThresholds": {
    "pendingEntryDays": 1,
    "fcr": 1.68,
    "mortalityPercent": 1
  },
  "financialConfig": {
    "supervisorCanAddFarmerExpense": true,
    "supervisorCanAddCompanyExpense": true,
    "farmerExpenseRequiresApproval": true
  }
}
```

5.12 Subscriptions

GET /api/v1/subscriptions/current

Purpose: Fetch the latest subscription state for the organization, including submitted payments.

Implement in UI: Billing status card / paywall

Auth required: Yes

Allowed roles: OWNER, SUPERVISOR, FARMER

`Sample request line: GET /api/v1/subscriptions/current`

Note: This endpoint can return null if no subscription record exists.

Note: Use it to decide whether to show trial, expired, grace, or payment-pending banners.

Dummy success response

```
{
  "id": "99999999-9999-9999-9999-999999999999",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "planId": "aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa",
  "planCode": "BASIC_MONTHLY",
  "planName": "Basic Monthly",
  "status": "PENDING_APPROVAL",
  "startsAt": "2026-05-12",
  "endsAt": "2026-06-03",
  "amountInr": 299,
  "upiDeepLink": "upi://pay?pa=boilerfarm@upi&pn=Boiler%20Farm%20Management&am=299",
  "notes": "Requested by Gopal Mewada",
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z",
  "payments": [
    {
      "id": "dddddddd-dddd-dddd-dddd-dddddddddddd",
      "organizationId": "22222222-2222-2222-2222-222222222222",
      "subscriptionId": "99999999-9999-9999-9999-999999999999",
      "amountInr": 299,
      "status": "SUBMITTED",
      "referenceNumber": "UPI-REF-20260504-01",
      "payerName": "Gopal Mewada",
      "payerPhone": "9876543210",
      "proofUrl": "https://cdn.example.com/payments/proof-01.jpg",
      "submittedAt": "2026-05-12T07:18:15.474Z",
      "reviewedAt": null,
      "reviewNote": null,
      "createdAt": "2026-05-12T07:18:15.474Z",
      "updatedAt": "2026-05-12T07:18:15.474Z"
    }
  ]
}
```

POST /api/v1/subscriptions/payments

Purpose: Submit the payment proof/reference after the owner pays via UPI.

Implement in UI: UPI payment confirmation

Auth required: Yes

Allowed roles: OWNER

`Sample request line: POST /api/v1/subscriptions/payments`

Note: Store the proof URL returned by your upload flow and send it here.

Note: This marks the payment as SUBMITTED for back-office approval.

Dummy request body

```
{
  "subscriptionId": "99999999-9999-9999-9999-999999999999",
  "referenceNumber": "UPI-REF-20260504-01",
  "payerName": "Gopal Mewada",
  "payerPhone": "9876543210",
  "proofUrl": "https://cdn.example.com/payments/proof-01.jpg"
}
```

Dummy success response

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "subscriptionId": "99999999-9999-9999-9999-999999999999",
```

```
  "amountInr": 12500,
  "status": "EXPIRED",
  "referenceNumber": "UPI-REF-20260504-01",
  "payerName": "Gopal Mewada",
  "payerPhone": "9876543210",
  "proofUrl": "https://cdn.example.com/payments/proof-01.jpg",
  "submittedAt": "2026-05-12T07:18:15.474Z",
  "reviewedAt": "2026-05-12T07:18:15.474Z",
  "reviewNote": "sample",
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z"
}
```

GET /api/v1/subscriptions/plans

Purpose: Manage low-cost subscription selection and UPI-based payment confirmation.

Implement in UI: Subscription / billing

Auth required: Yes

Allowed roles: OWNER, SUPERVISOR, FARMER

`Sample request line: GET /api/v1/subscriptions/plans`

Note: These endpoints are only needed for billing-related screens.

Note: The business workflow is plan selection -> request -> UPI payment -> proof submission.

Dummy success response

```
[
  {
    "id": "aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa",
    "code": "TRIAL_14_DAYS",
    "name": "Trial 14 Days",
    "amountInr": 0,
    "durationDays": 14,
    "maxFarms": null,
    "maxUsers": null,
    "description": "Short onboarding trial for new farm owners.",
    "isActive": true
  },
  {
    "id": "cccccccc-cccc-cccc-cccc-cccccccccccc",
    "code": "BASIC_MONTHLY",
    "name": "Basic Monthly",
    "amountInr": 299,
    "durationDays": 30,
    "maxFarms": 5,
    "maxUsers": 10,
    "description": "Low-cost starter plan for broiler farm operations.",
    "isActive": true
  }
]
```

POST /api/v1/subscriptions/requests

Purpose: Create a subscription request and return a UPI deep link for manual payment.

Implement in UI: Choose plan > confirm

Auth required: Yes

Allowed roles: OWNER

`Sample request line: POST /api/v1/subscriptions/requests`

Note: Only OWNER can create or change the subscription request.

Note: The response includes upiDeepLink for direct UPI launch from Android.

Dummy request body

```
{
  "planCode": "BASIC_MONTHLY"
}
```

Dummy success response

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "organizationId": "22222222-2222-2222-2222-222222222222",
  "planId": "aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa",
  "planCode": "BASIC_MONTHLY",
  "planName": "Sample Name",
  "status": "ACTIVE",
  "startsAt": "2026-05-12",
  "endsAt": "2026-05-12",
  "amountInr": 12500,
  "upiDeepLink": "upi://pay?pa=boilerfarm@upi&pn=Boiler%20Farm%20Management&am=299",
  "notes": "Captured from mobile app",
  "createdAt": "2026-05-12T07:18:15.474Z",
  "updatedAt": "2026-05-12T07:18:15.474Z",
  "payments": [
    {
      "id": "11111111-1111-1111-1111-111111111111",
      "organizationId": "22222222-2222-2222-2222-222222222222",
      "subscriptionId": "99999999-9999-9999-9999-999999999999",
      "amountInr": 12500,
      "status": "EXPIRED",
      "referenceNumber": "UPI-REF-20260504-01",
      "payerName": "Gopal Mewada",
      "payerPhone": "9876543210",
      "proofUrl": "https://cdn.example.com/payments/proof-01.jpg",
      "submittedAt": "2026-05-12T07:18:15.474Z",
      "reviewedAt": "2026-05-12T07:18:15.474Z",
      "reviewNote": "sample",
      "createdAt": "2026-05-12T07:18:15.474Z",
      "updatedAt": "2026-05-12T07:18:15.474Z"
    }
  ]
}
```

5.13 Support

GET /docs

Purpose: Open the interactive Swagger UI that mirrors the backend contract used in this handoff document.

Implement in UI: Developer / QA reference only.

Auth required: No

Allowed roles: Public

`Sample request line: GET /docs`

Note: This route returns HTML, not JSON.

Note: Frontend developers can use it to inspect schemas and test secured endpoints manually.

Dummy success response

```
HTML Swagger UI page
```

GET /health

Purpose: Basic operational health check for app bootstrap diagnostics, uptime checks, and Railway monitoring.

Implement in UI: Not a UI screen; useful for app diagnostics and deployment monitoring.

Auth required: No

Allowed roles: Public

`Sample request line: GET /health`

Note: Returns service name, environment, uptime, and server timestamp.

Note: Can be used by the mobile app to detect whether the API is reachable.

Dummy success response

```
{
  "name": "Boiler Farm Management API",
  "environment": "development",
  "uptimeSeconds": 324,
  "timestamp": "2026-05-12T07:18:15.474Z"
}
```

6. Standard Error Shapes

Validation error (HTTP 422)

```
{
  "message": "Validation failed",
  "details": {
    "fieldName": {
      "message": "Required"
    }
  }
}
```

Application / business rule error (HTTP 400 / 403 / 404 / 409)

```
{
  "message": "User not found",
  "details": null
}
```

Unhandled server error (HTTP 500)

```
{
  "message": "Internal server error"
}
```

7. Enum Reference

Enum	Allowed values
BatchStatus	ACTIVE, CANCELLED, PLANNED, SALES_RUNNING, SETTLEMENT_PENDING, CLOSED
CatalogItemType	CHICKS, FEED, MEDICINE, VACCINE, EQUIPMENT, OTHER
CommentTargetType	PURCHASE, SETTLEMENT, FARM, BATCH, DAILY_LOG, TREATMENT, COST, SALE, PAYMENT
ExpenseApprovalStatus	PENDING, APPROVED, REJECTED
ExpenseCategoryCode	CHICKS, FEED, MEDICINE, VACCINE, TRANSPORT, OFFICE_EXPENSE, SUPERVISOR_EXPENSE, OTHER_COMPANY, ELECTRICITY, COCO_PITH, LABOUR, WATER, DIESEL, SHED_MAINTENANCE, REPAIRS, MISCELLANEOUS, OTHER_FARMER
ExpenseLedger	FARMER, COMPANY
FarmStatus	ACTIVE, INACTIVE
FinanceEntryType	INVESTMENT, OTHER_INCOME, OTHER_EXPENSE
InventoryMovementType	PURCHASE, ALLOCATION, ADJUSTMENT, RETURN
NotificationSeverity	INFO, WARNING, CRITICAL
NotificationType	MORTALITY_ALERT, FEED_ALERT, VACCINE_DUE, FCR_ALERT, PENDING_ENTRY, SALES_READY, PAYMENT_DUE, GENERAL
PaymentDirection	INBOUND, OUTBOUND
PaymentEntryType	OTHER, PURCHASE, EXPENSE, SALE_RECEIPT, SETTLEMENT, INVESTMENT
PaymentStatus	EXPIRED, PENDING, SUBMITTED, APPROVED, REJECTED
PayoutUnit	PER_BIRD_PLACED, PER_BIRD_SOLD, PER_KG_SOLD
PurchaseType	CHICKS, FEED, MEDICINE, VACCINE, EQUIPMENT, OTHER
SaleStatus	CANCELLED, DRAFT, CONFIRMED

SettlementStatus	DRAFT, FINALIZED
SubscriptionStatus	ACTIVE, TRIAL, GRACE, EXPIRED, PENDING_APPROVAL, CANCELLED
TransactionPaymentStatus	CANCELLED, PENDING, PARTIAL, PAID
TreatmentKind	OTHER, VACCINATION, MEDICATION
UserRole	OWNER, ACCOUNTS, SUPERVISOR, FARMER
UserStatus	ACTIVE, INVITED, DISABLED

8. Frontend Build Notes

Use GET /auth/me after login or refresh to hydrate the current user, biometric flag, and permission set before showing role-specific tabs.

Store refresh tokens securely and use POST /auth/refresh for silent session renewal instead of redirecting to login too aggressively.

Use /expenses as the canonical expense module and keep /costs only for backward compatibility or legacy clients.

Do not recompute settlement, payment status, or P&L on the client when the backend already returns those values.

For exports, call the endpoint as blob/binary and read the Content-Disposition header to preserve file names.

For farmer apps, rely on server-side visibility instead of trying to filter farm or batch access only on the client.

Use clientReferenceId on mobile capture flows when offline sync or duplicate-submit protection is required.